Research Progress Report

Exact-Statistics Anchoring for Generator-Based Heterogeneous Federated Learning under Long-Tailed Data

Why rare classes vanish from GeFL-F, and how Ours-A brings them back

Submitted by: [Student Name] · [Roll No.]

Supervisor: [Professor Name]

[Department], [Institute]

Version 2 · 15 October 2026

Summary

- **Problem:** under long-tailed clients GeFL-F loses 21–25 points; rare classes are almost never predicted.
- **Cause:** FedAvg with Adam's weight decay shrinks the generator's class rows of rare classes, even when 70–90 % of the clients hold them (Theorem 1, measured on the real model).
- **Our architecture (Ours-A):** class identity from the exact class mean, fake features corrected to exact class statistics, consensus labels, logit adjustment (a loss that cancels each client's own class skew).

Result (3 seeds, same seeds and splits for every method)	GeFL-F	Ours-A	Gain
MNIST, long tail (balanced accuracy)	75.25	**94.82**	+19.6
FashionMNIST, long tail	58.47	**79.67**	+21.2
CIFAR-10, long tail	33.74	**44.38**	+10.6
SVHN, long tail	50.49	**54.41**	+3.9
SVHN, long tail, with the automatic switch to class rows (Part 5.7)	50.49	**65.60**	+15.1
MNIST, 100 clients, long tail	75.98	**94.30**	+18.3
FashionMNIST, 50 / 100 clients, long tail	61.9	**79.5 / 79.0**	+17.6 / +17.1
Paper's balanced setting vs the best of its 10 methods	—	above on MNIST (10/50/100 clients), CIFAR-10 and SVHN (with the switch); level on FashionMNIST	

Honest comparison point (new in this version, Part 6)

Most of GeFL-F's long-tail loss sits in the **last layer** of each head. Re-fitting only that layer on real data (a known fix: CCVR, CReFF, Fed3R) lifts GeFL-F to 90.7 / 79.2 / 64.8 (MNIST / FashionMNIST / SVHN, long tail).

Ours-A still beats this stronger baseline (94.8 / 79.7 / 65.6), but by **+4.1 / +0.5 / +0.8**, not +20. The paper must report it.

What changed since version 1

Collapse theorem corrected (Part 2.2): the old "fewer than half the clients" rule was wrong in our own data. New exact bound (Theorem 1) and the role of Adam's decay (Lemma 2), checked on the real model.

New Part 6: where the remaining error lives (the last layer), a federated fix from exact statistics (exact readout), and the stronger baseline above.

New Part 7: why each failure happens and why each fix works, step by step, with the evidence and what we would have seen if the explanation were wrong.

New results: leave-one-out ablation (Part 5.6), SVHN switch (Part 5.7), FashionMNIST at 50 / 100 clients (Part 5.5), covariance correction (Part 8).

Claims made more precise: which parts of Ours-A are new and which are known techniques (Parts 4.1, 4.8).

How to read the numbers

This report uses many measurements. Only four of them decide whether the method works; one exists only to compare with the original paper; the rest are *diagnostics*: they explain why something works or fails (Part 7 walks through each failure), and are not goals in themselves. Every accuracy is in percent; a difference between two accuracies is in *points* (79.7 − 58.5 = +21.2 points).

A. The numbers that decide whether the method works

Measurement	What it measures, in plain words	How to read it
Balanced accuracy	Accuracy on each of the 10 classes separately, then averaged, so a rare class counts as much as a common one. Measured on the test set after the last training round.	10 = guessing, 100 = perfect. **The headline number of this report.**
Rare-class accuracy ("tail recall")	The same, but averaged over the 4 rarest classes only.	Shows whether the rare classes came back. Close to balanced accuracy = rare classes are no longer neglected.
Gain (points)	Accuracy of one method minus another, on the same data splits and starting weights.	+21 = 21 percentage points better.
± and p-value	Every result is repeated 3 times with different random splits ("seeds"); ± is the spread over the 3. The p-value asks: if the two methods were really equal, how likely is a difference this consistent?	± small compared with the gain = trustworthy. p below 0.05 is the usual bar. With only 3 repeats, p-values are rough.

B. Only for comparison with the original paper

Measurement	What it measures, in plain words	How to read it
Paper's accuracy (best_mean_acc)	For each of the 10 model types, its best test accuracy over all rounds; then the average of the 10.	Optimistic, because it picks the best round by looking at the test set. We use it only where we compare with the paper's own tables; our own claims use balanced accuracy at the last round.

C. Diagnostics: why it works or fails

Measurement	What it measures, in plain words	How to read it
Row length ratio (rare / common)	The original generator stores each class as a row of 512 numbers. We compare the length of the rows of rare classes with those of common classes.	1 = rare classes treated like common ones. 0.2 = rare-class rows shrank to a fifth: the generator can no longer tell rare classes apart. **Proof of the collapse** (Part 2).
Holder gain / non-holder loss, and the threshold θ	Per round: how much the clients that have a class lengthen its row, against how much the clients without it shorten it. θ is the share of clients that must have the class for the row not to shrink.	Measured θ = 0.61–0.93: a rare class shrinks unless 61–93 % of clients have it, not 50 % as we first claimed.
Fake-sample correctness ("fidelity")	A separate judge classifier, trained only on real data and never used in training, labels the generator's fake samples of a rare class. Share it labels correctly.	10 % = fakes look like random classes. Ceiling = the judge's score on real data (97 % MNIST, 84 % FashionMNIST).
Spread ratio	How varied the fake samples of a class are, divided by how varied the real ones are.	1 = right variety. 0.45 = fakes too similar to each other.
Fake-to-real distance (maximum mean discrepancy)	One number for how different the whole cloud of fake features is from the cloud of real features.	0 = identical; lower is better. Compare only within one dataset.

Measurement	What it measures, in plain words	How to read it
Train-on-fake, test-on-real accuracy	Train a classifier only on fake features, test it on real data.	The most direct test of whether fake features are useful.
Class separability (F) and **nearest-mean accuracy**	Can the *average* feature of each class tell the classes apart? F: distance between class averages relative to the spread inside a class. Nearest-mean accuracy: label each real sample by the closest class average.	F near 0 and nearest-mean accuracy near 10 % = class averages are almost identical. Explains why SVHN fails (12 %).
Last-layer gap ("readout gap")	Accuracy after re-fitting only each model's last layer on pooled real data, minus the accuracy as trained.	How much accuracy is lost in the last layer alone. GeFL-F loses 14–20 points there; Ours-A 0.1–2 (Part 6).
Class-share correlation	Correlation between how often a client sees each class and how often its model predicts it.	0 = predictions not biased toward the client's own common classes. Used to show what logit adjustment does.
Total-variation distance of labels	How far the soft training labels are from a real-data judge's answers.	0 = same answers. Used to show what consensus labels do.
Privacy budget ε	Strength of the formal privacy guarantee when noise is added to the uploaded class sums.	Smaller = more private. 8 = moderate, 2 = strong.

D. The settings

Measurement	What it measures, in plain words	How to read it
Long tail (imbalance 100 : 1)	The most common class has 100 times as many samples as the rarest.	The setting this work is built for.
Balanced setting	Every class has the same number of samples, split evenly over clients: the original paper's setting.	Checks that the method does no harm when there is no imbalance.
Dirichlet 0.5	How unevenly each class is shared among clients. Smaller = more uneven.	At 0.5 most clients have several classes missing entirely.
Number of clients (10, 50, 100)	Devices taking part. There are always 10 model types, so 50 clients means 5 clients per model type.	More clients = less data each.

If you remember only four numbers

Balanced accuracy (does it work), **rare-class accuracy** (do rare classes come back), **row length ratio** (proof of the cause), and **last-layer gap** (where the remaining error is).

Short names used in tables: Ours-A = our final method; GeFL-F = the original paper's method; holder-weighted averaging, moment correction, sample selection, consensus labels and logit adjustment are the parts described in Part 4.

1. The baseline: GeFL-F and the full lifecycle of its data

1.1 Setting and notation

- **Clients.** K clients (10 clients, 50 or 100). Client k holds private data D_k and a classifier head of architecture a(k), one of G = 10 CNNs (CNN-1 … CNN-10), assigned contiguously (K/10 clients per architecture).
- **Data.** MNIST, FashionMNIST and SVHN use a 10% fraction of the training set (6,000 / 6,000 / 7,326 images); CIFAR-10 uses 50% (25,000). In the paper's setting the data are split evenly (balanced) across clients. Test sets are the full balanced test sets.
- **Models.** One shared feature extractor φ, one shared conditional generator G(h | y), and K private heads h_k.
- **Metric.** The paper's *best_mean_acc*: each architecture's test accuracy at its best round, averaged over the 10 architectures. Under the long tail we report balanced accuracy at the final round, and *rare-class accuracy* (average accuracy on the four rarest classes; see "How to read the numbers").

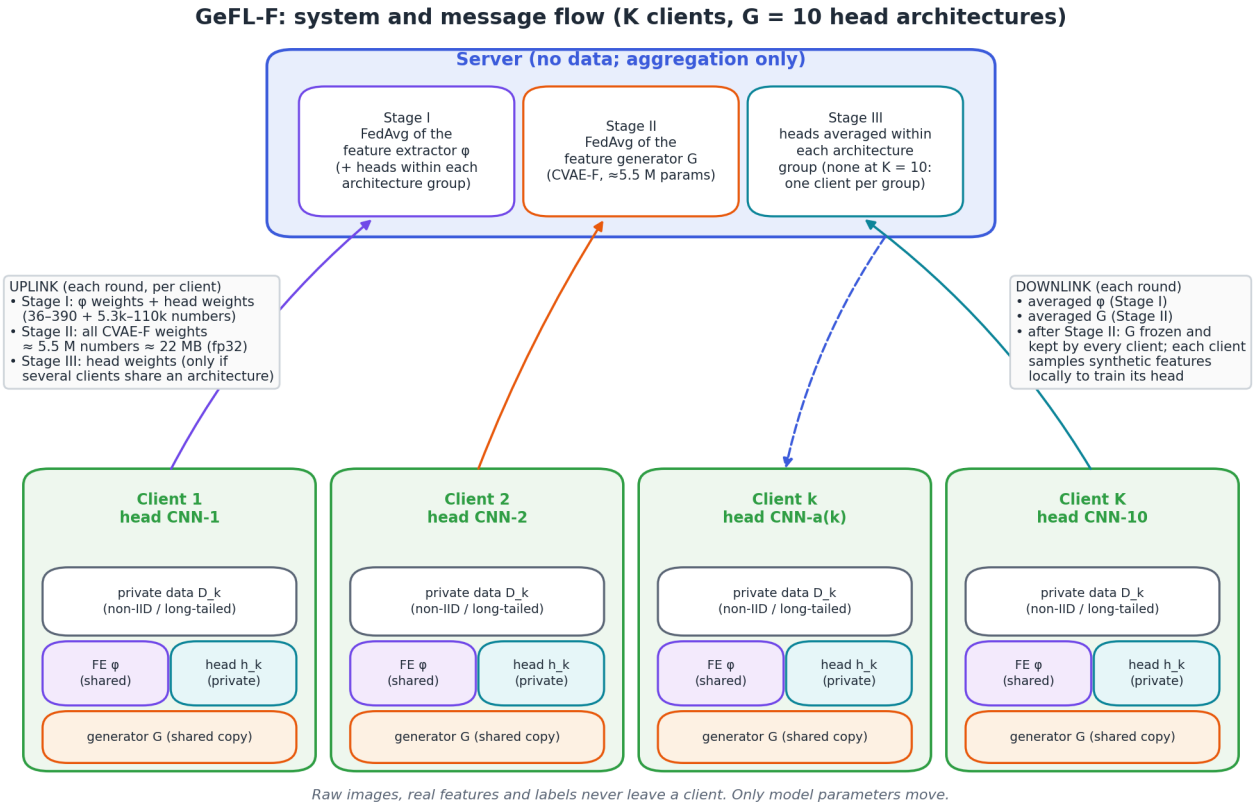


Figure 1. GeFL-F system view: what is local, what is shared, and which messages travel in each stage.

1.2 Model architectures and tensor shapes

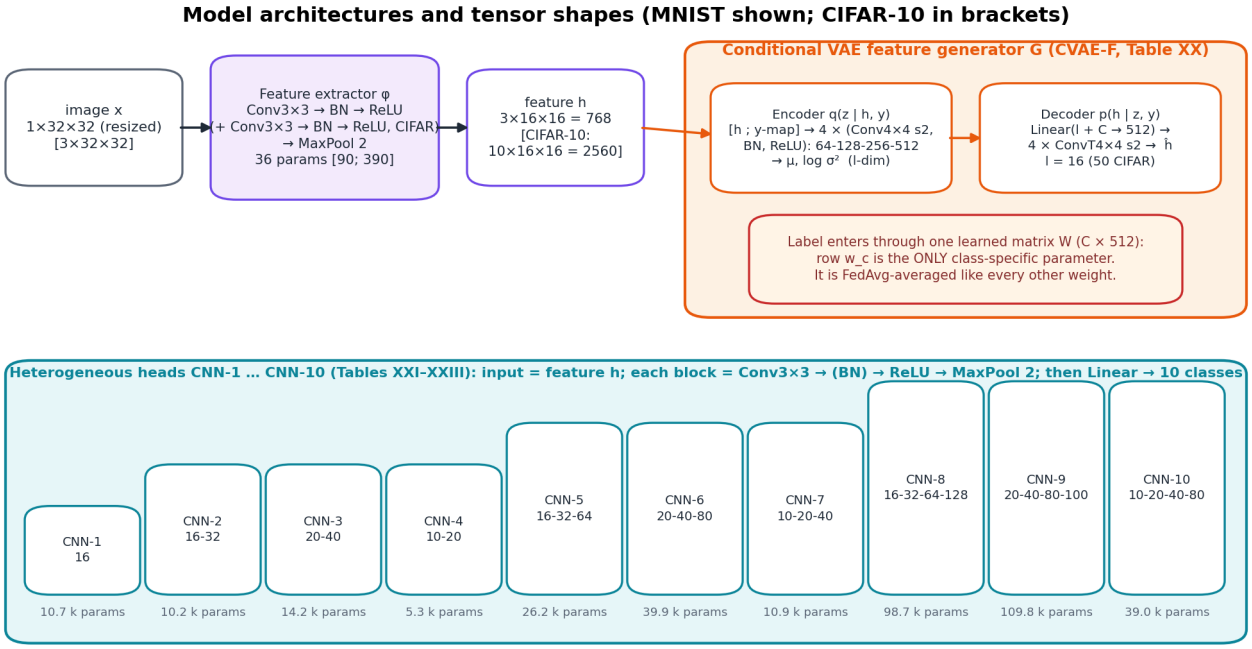
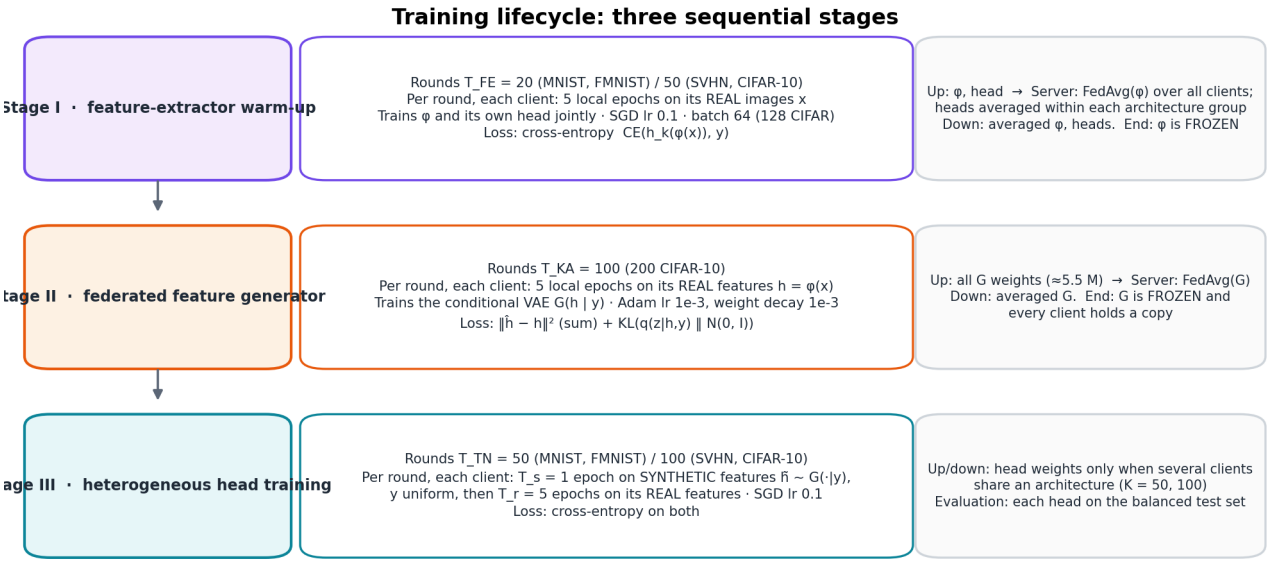


Figure 2. Architectures. The original generator's label projection W (C × 512) holds the only class-specific parameters; this matters in Part 2.

Model	Input	Output	Parameters	Shared?
Feature extractor φ (Table XIV)	image resized to 1×32×32 (MNIST, FashionMNIST) / 3×32×32	feature h: 3×16×16 = 768 (MNIST, FashionMNIST, SVHN); CIFAR-10 10×16×16 = 2,560	36 / 90 / 390	yes (FedAvg in Stage I, then frozen)
Generator G, original generator (Table XX)	h (+ label map) → z (l = 16; 50 for CIFAR-10)	ĥ, same shape as h	≈ 5.5 M	yes (FedAvg in Stage II, then frozen)
Heads CNN-1 … CNN-10 (Tables XXI–XXIII)	feature h	10 logits	5.3 k – 109.8 k	no (averaged only within an architecture group)

1.3 Lifecycle: three sequential stages



Metric of the paper: best_mean_acc = test accuracy of each architecture at its best round, averaged over the 10 architectures.

Figure 3. The three stages, with rounds, local epochs, data, optimiser and loss of each.

Stage I: feature-extractor warm-up (T_FE = 20 rounds; 50 for SVHN and CIFAR-10)

- **Local data:** the client's real images x with labels y.
- **Local training:** 5 epochs per round; φ and the client's head trained jointly; stochastic gradient descent, learning rate 0.1, batch 64 (128 for CIFAR-10); loss = cross-entropy cross-entropy(h_k(φ(x)), y).
- **Uplink:** φ's weights (36–390 numbers) and the head's weights (5.3 k–110 k numbers).
- **Aggregation:** φ is averaged over all clients (FedAvg, equation below); heads are averaged within each architecture group.

$$\theta^{(r+1)} = \sum_{k=1}^{K} \frac{n_k}{n} \theta_k^{(r)}, \qquad W_c^{(r+1)} = \frac{1}{K} \sum_{k} W_{c,k}^{(r)} \quad \text{(flat)} \tag{1}$$

- **Downlink:** the averaged φ and heads. At the end of Stage I, φ is frozen for the rest of training.

Stage II: federated feature generator (T_KA = 100 rounds; 200 for CIFAR-10)

- **Local data:** real features h = φ(x) of the client's own images, with their labels.
- **Local training:** 5 epochs per round of the conditional variational autoencoder, Adam with learning rate 10^{-3} and coupled weight decay 10^{-3}, batch 64. The loss is the evidence lower bound:

$$\mathcal{L}_G = \|\hat{h} - h\|_2^2 + \mathrm{KL}(q(z \mid h, y) \parallel \mathcal{N}(0, I)), \qquad \hat{h} = \mathrm{dec}(z,\ W^\top e_y) \tag{2}$$

- The label enters the decoder through W^T e_y, i.e. the class-c row w_c of the label projection: these C × 512 numbers are the generator's *only* class-specific parameters.
- **Uplink:** every generator weight, about 5.5 million numbers (≈ 22 MB at fp32) per client per round, ≈ 2.2 GB per client over Stage II.
- **Aggregation:** FedAvg of all weights, including W (equation 1). **Downlink:** the averaged generator. At the end of Stage II, G is frozen and every client keeps a copy.

Stage III: heterogeneous head training (T_TN = 50 rounds; 100 for SVHN and CIFAR-10)

- **Synthetic phase:** 1 synthetic epoch epoch on generated features h̃ = G(z, y), with y drawn uniformly over the 10 classes and z ~ N(0, I), sampled locally from the client's own copy of G; loss cross-entropy(h_k(h̃), y).

- **Real phase:** T_r = 5 epochs on the client's real features φ(x); loss cross-entropy(h_k(φ(x)), y); stochastic gradient descent, learning rate 0.1.
- **Uplink and aggregation:** head weights, averaged within an architecture group only when several clients share an architecture (50 clients, 100). At 10 clients each architecture has one client, and nothing is exchanged in this stage.

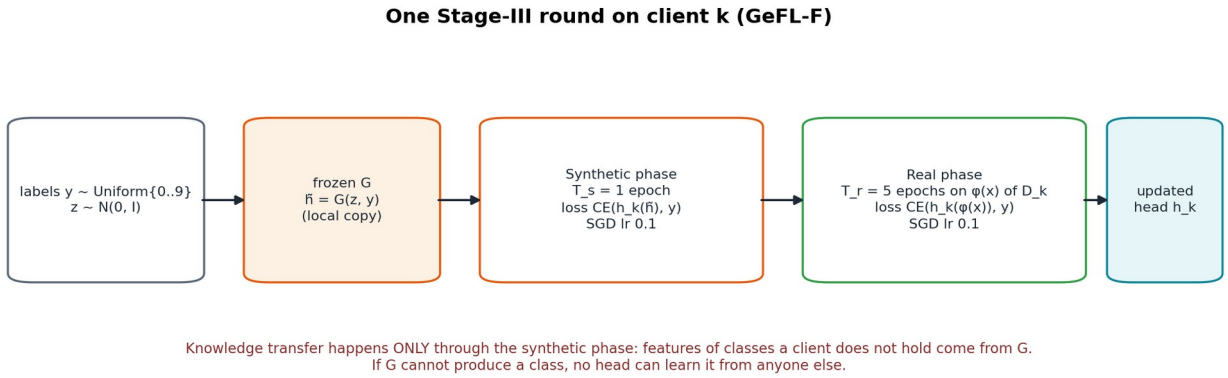


Figure 4. One Stage-III round. Classes a client does not hold reach its head only through the generator.

1.4 Communication summary

Stage	Uplink per client per round	Server operation	Downlink
I	φ (36–390) + head (5.3 k–110 k numbers)	FedAvg(φ); heads within groups	averaged φ, heads
II	all generator weights, ≈ 5.5 M numbers (≈ 22 MB)	FedAvg(G)	averaged G
III	head weights (only if an architecture has several clients)	average within architecture groups	averaged heads

Privacy: images, features and labels never leave a client; only model weights move (same as FedAvg).

1.5 Validation of our reproduction

Setting	Paper	Authors' code	Ours
MNIST balanced, 10 clients (best_mean_acc)	95.47	96.07	96.02 ± 0.04
MNIST balanced, 50 clients	95.04	–	95.16
MNIST balanced, 100 clients	94.63	–	94.61
FashionMNIST balanced, 10 clients	83.14	–	82.67 ± 0.31
MNIST long tail, seed 0 (final balanced accuracy)	–	75.84	73.92

Our GeFL-F matches the paper within 0.5 points in the balanced setting.

2. The problem: rare classes vanish under long-tailed clients

2.1 The long-tailed federation

The most common class has 100 times as many training samples as the rarest, with sizes falling evenly (on a log scale) in between (MNIST: about 2,400 down to 24 samples). Each class is shared out among the clients with a Dirichlet(0.5) draw, so most clients have many samples of a few classes and none of several others. Test sets stay balanced.

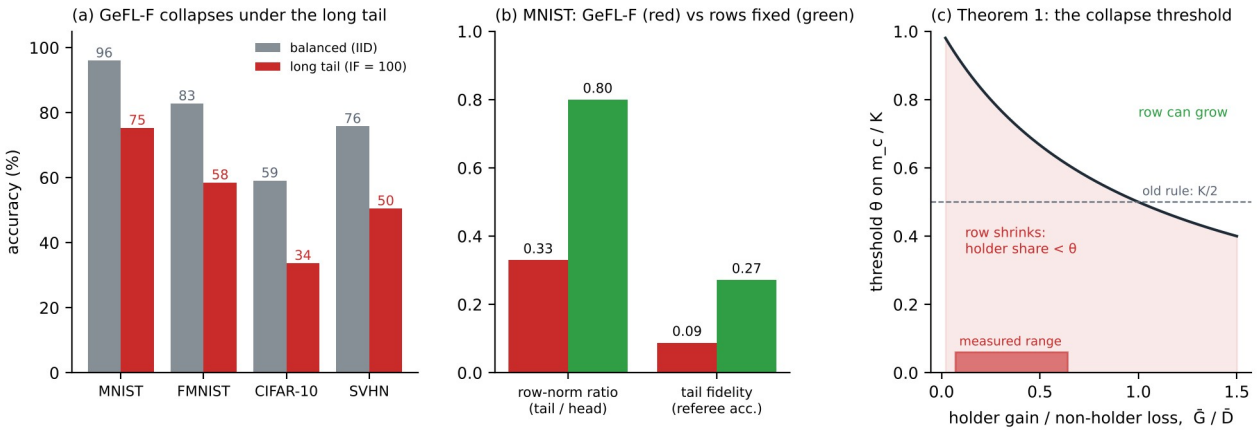


Figure 5. (a) GeFL-F's accuracy under the long tail. (b) Its tail classes' conditioning rows and fidelity. (c) The collapse threshold of Theorem 1 (Part 2.2).

2.2 Diagnosis: the conditioning-row recursion

Write r_c for the generator's row of class c. In one round, m_c of the K clients hold class c and K − m_c do not. A holder's local training changes the row's length by +G_k (data pull it outward); a non-holder sees no class-c data, only the weight-decay gradient, and shortens it by D_k. FedAvg then takes the plain average of the K rows. With p_c = m_c / K, Ḡ_c the average holder gain and D̄_c the average non-holder loss:

$$\|r_c^+\| \;\le\; \|r_c\| + p_c\,\bar{G}_c - (1-p_c)\,\bar{D}_c, \qquad \text{shrinks whenever } \; p_c = \frac{m_c}{K} < \theta_c = \frac{\bar{D}_c}{\bar{G}_c + \bar{D}_c}$$

(Theorem 1)

- **Theorem 1 (exact, no assumption on the optimiser).** The row's length after averaging is at most the right-hand side, with equality when all client rows point the same way. It shrinks whenever the holder share is below θ_c.
- **Lemma 2 (why the loss D̄ is large).** Adam divides the gradient by its own running size, so the decay gradient λ·r_c becomes a step of about the learning rate η per coordinate, **whatever λ is** (10^{-4} or 10^{-3} give the same step). Only λ = 0 removes it. With plain stochastic gradient descent the same decay would be about 1,000 times weaker here.
- **Consequence.** Rare classes appear in few batches, so their holders gain little (Ḡ small) while non-holders lose a full step: θ_c is close to 1, not ½. The version-1 rule "rows shrink when fewer than half the clients hold the class" is the special case Ḡ = D̄, and it is too optimistic.

Figure 5(c) plots θ against Ḡ / D̄. Measured on MNIST long tail: Ḡ / D̄ = 0.07–0.64, so θ = 0.61–0.93; the K/2 rule would predict no collapse at all.

2.3 Evidence

Measurement (imbalance 100 : 1, Dirichlet 0.5, 10 clients)	MNIST	FashionMNIST	SVHN
Tail-to-head row-norm ratio, GeFL-F	0.33	0.22	0.18–0.49
Same, with non-holders excluded from the average	0.80	1.01	0.96–1.05

Measurement (imbalance 100 : 1, Dirichlet 0.5, 10 clients)	MNIST	FashionMNIST	SVHN
Tail fidelity of GeFL-F's generator (referee accuracy on synthetic tail features)	8.8 %	19.6 %	–
GeFL-F balanced accuracy: balanced → long tail	96.0 → 75.3	82.8 → 58.5	75.8 → 50.5

Ratio far below 1 under the long tail, and restored when non-holders are removed from the average.

Direct test of Theorem 1 on the real generator (T01: MNIST long tail, 10 clients, 40 rounds, 4 arms)

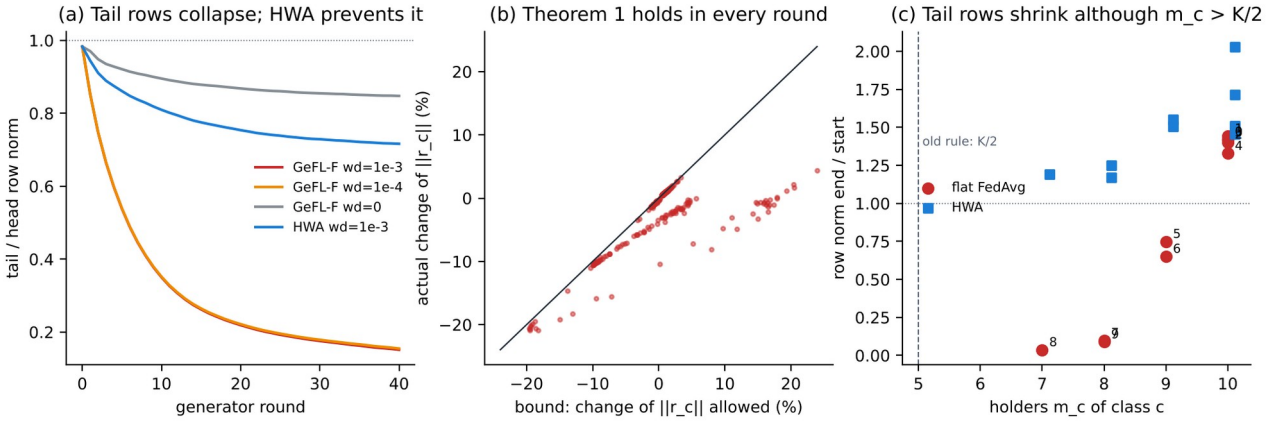


Figure 5b. (a) Tail rows shrink to 15 % of head rows with decay 10^{-3} or 10^{-4} (identical curves), not without decay, and not with holder-weighted averaging. (b) The bound holds in all 1,200 row-rounds. (c) Rows of classes held by 7–9 of 10 clients still collapse; under holder-weighted averaging they grow.

Arm (same data, same seed)	Tail / head row length at round 40	What it shows
GeFL-F, decay 10^{-3} (paper's setting)	0.152 (tail rows at 4–10 % of start)	collapse although every class has ≥ 7 of 10 holders
GeFL-F, decay 10^{-4}	0.155	the size of the decay does not matter (Lemma 2)
GeFL-F, decay 0	0.848	only the presence of decay matters
Holder-weighted averaging, decay 10^{-3}	tail rows grow to 1.17–1.25×	averaging over holders only removes the $\bar{D}$ term

Is this new? The collapse of class-specific weights under missing classes is known (FedRS, FedLC, FedAwS for classifier layers). What we add is the exact bound for a *generator's* conditioning rows and the Adam-decay explanation. The obvious existing fixes (AdamW, no decay on the rows, lazy decay) are compared with holder-weighted averaging in the prepared job K15 (Part 8).

2.4 Why this breaks knowledge transfer

A head trained on fake features is only as good on real data as the fake features are close to real ones (total-variation distance). For rare classes this gap is large:

$$R_{real}(h) \;\le\; R_{syn}(h) + \ell_{\max} \cdot \mathrm{TV}(p_{syn},\; p_{real}) \qquad (3)$$

3. The research journey: every approach we tested

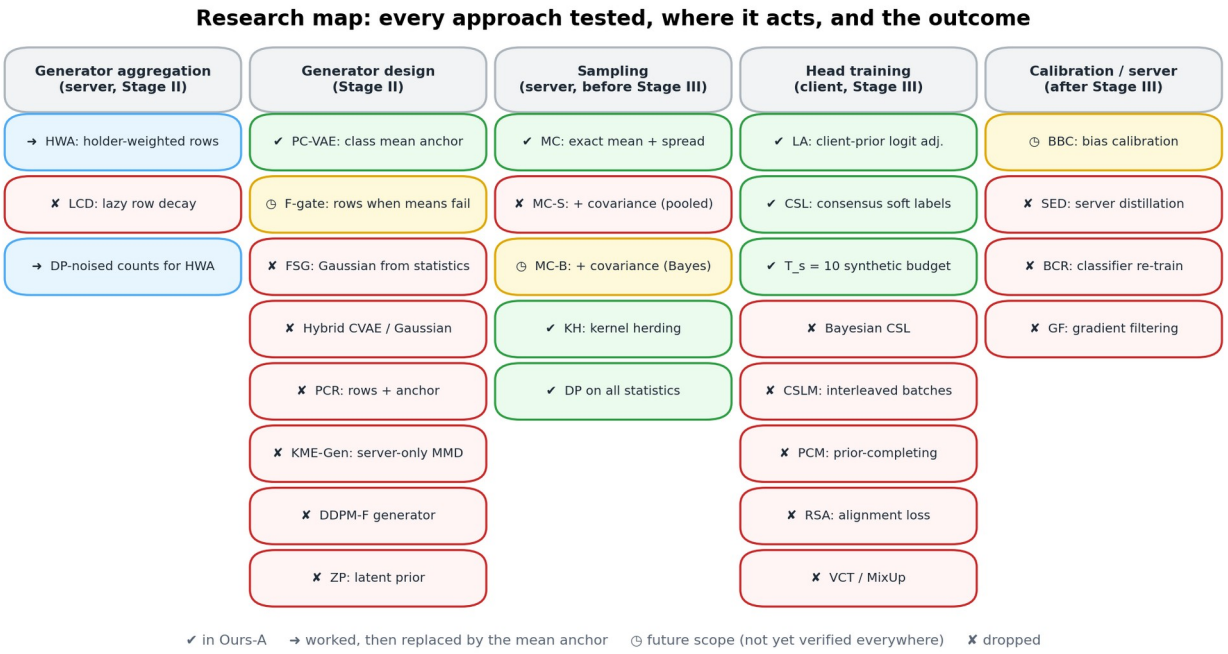


Figure 6. Research map. Green: kept in the final method. Red: tested and dropped on the evidence.

3.1 Repairing the generator's aggregation

Holder-weighted averaging (kept)

Where: server, Stage II, class rows only. *How:* weight each client's row by its effective sample count (Cui et al., CVPR 2019); non-holders get weight 0. This is an established re-weighting, applied here to the generator's rows; it removes the $\bar{D}$ term of Theorem 1:

$$w_c \leftarrow \frac{\sum_k E(n_{kc})\, w_c^{(k)}}{\sum_k E(n_{kc})}, \qquad E(n) = \frac{1-\beta^n}{1-\beta}, \quad \beta = 0.999 \tag{4}$$

Result: +9.0 / +11.2 / +3.4 / +6.0 (MNIST / FashionMNIST / CIFAR-10 / SVHN, long tail); row ratio back to 0.80–1.05. Later replaced in Ours-A by the mean anchor, which has no class rows at all.

Lazy decay (dropped)

Where: clients, Stage II. *How:* update and decay only rows of classes present in the batch. *Result:* 78.6 vs 79.1 for holder-weighted averaging; adds nothing.

3.2 Training the heads

Logit adjustment with the client's own class shares (kept; established technique)

Where: each client, real-data phase. *How:* Menon et al. (ICLR 2021) with the client's own private prior π_k:

$$\ell_{LA}(h, y) = -\log \frac{\exp(f_y(h) + \tau \log \pi_{k,y})}{\sum_j \exp(f_j(h) + \tau \log \pi_{k,j})} \tag{5}$$

Result: +3.6 to +6.3 on all four datasets. Needed because the 5 real epochs pull each head back to its own skew. No effect in balanced.

Consensus labels on synthetic features (kept)

Where: synthetic phase; clients send only prediction sums on a shared fake pool. *How:* target = mix of the label and the average prediction of all architectures (FedMD, FedDF):

$$t(\tilde{h}) = (1-\beta)\, e_y + \beta \frac{1}{G} \sum_{g=1}^{G} \mathrm{softmax}\big(h_g(\tilde{h})\big), \qquad \beta = 0.5 \tag{6}$$

Result: +2.5 MNIST long tail; +0.5 to +0.8 balanced. β = 0.5 better than β = 1.

10 synthetic epochs: a larger synthetic budget (kept)

Why (Proposition 5): more fake training helps only when the generator is faithful (low label bias b).

$$\mathcal{E}(w) = \frac{\sigma^2(1-w)}{n_r} + w^2 b^2, \qquad w^\star = \frac{\sigma^2}{2\, n_r\, b^2} \quad \text{(optimal synthetic share falls with label bias } b\text{)}$$

(Proposition 5)

Result: with the anchored generator +1.7 (MNIST long tail) and +3.9 at 100 clients; with the original generator −0.05, as predicted.

Dropped head-side idea	Placement and model	Result	Why it failed
Bayesian consensus labels	Replace the fixed mixture by a Bayes posterior under symmetric generator noise, fidelity $\hat{\rho}$ estimated from ensemble agreement	83.16 vs 84.22 (FashionMNIST balanced, p = 0.004); 87.3 vs 90.5 (MNIST long tail)	$\hat{\rho}$ is circular: heads trained on the generator agree with it ($\hat{\rho} \to 0.999$), so the target collapses to the hard label
Consensus labels inside real batches	Interleave consensus-labelled synthetic features into every real batch	SVHN long tail 49.9 vs 62.4	Mixes the client-prior logit adjustment with balanced synthetic labels in one batch
Prior-completing batches	Fill each real batch's missing classes with synthetic samples	FashionMNIST long tail 59.6 vs 65.2 (with logit adjustment)	Needs faithful tail features, which do not exist before the generator is fixed
Real–synthetic alignment	Align head embeddings of synthetic and real features on held classes	synthetic–real gap grew on unheld classes	Fits the alignment to the held classes only
Our mixing variant / MixUp	Mix real features with consensus-labelled synthetic partners (our mixing variant) or with each other (MixUp)	CIFAR-10 balanced: our mixing variant 60.29, MixUp 59.98, neither 60.93; long tail tail recall +10 with our mixing variant	Feature-space mixing does not regularise past the frozen extractor (Part 5.4)

3.3 Server-side corrections

Approach	Placement and model	Result	Verdict
Server-side ensemble distillation	Server distils the heads' ensemble into each head on fresh synthetic features	FashionMNIST balanced 81.69 → 80.96; long tail 70.65 → 71.30	Dropped: bounded by equation 3; the ensemble's knowledge reaches heads only as faithfully as the generator allows
Balanced classifier re-training	Server re-fits each head's last layer on balanced synthetic features (CReFF-like)	62.0 vs 65.2 for logit adjustment (FashionMNIST long tail, quick)	Dropped: weaker than logit adjustment
Gradient filtering	Filter conflicting synthetic gradients	58.5 vs GeFL-F 58.8	Dropped: no effect

Approach	Placement and model	Result	Verdict
Bias calibration (Saerens et al., 2002)	Fit C logit offsets per head on balanced fake features	+0.35 to +1.4 under the long tail (MNIST/FashionMNIST); stays below the last-layer refit in every run	Kept as an optional, gated post-hoc step; overlaps with logit adjustment (Part 5.6); cannot fix a weight error (Part 6)

3.4 Alternative generators

Generator	Placement and model	Result	Verdict and reason
Gaussian generator from exact statistics	Server builds N(m_c, Σ_pooled) from secure-aggregated sums; no generator training	MNIST long tail 87.0; CIFAR-10 long tail 44.2; SVHN long tail 44.2 < GeFL-F 50.5	Dropped: its Bayes head is linear discriminant analysis (eq. 8), which reaches only 21 % on SVHN pixels
Hybrid original generator / Gaussian	Mixture of the two generators	87.1 vs 87.9 (full schedule; the quick pass had favoured it)	Dropped
Server-trained generator (kernel matching)	Generator trained only at the server, by fake-to-real distance to exact kernel embeddings	MNIST 89.9, FashionMNIST 75.1 (+15 over GeFL-F) at 1/3000 of the communication	Dropped as main method; noted as an efficient variant
Diffusion generator (paper's diffusion generator)	Our components on the paper's best feature generator	SVHN balanced 70.37 vs diffusion GeFL-F 69.77 (p = 0.10); far below 76–77 with original generator	Dropped: slow (hours per seed) and weaker here
Latent prior fitted afterwards	Fit z ~ N(m, S) to the encoder means (exact sums)	Mean anchor + latent prior 91.69 vs mean anchor + moment correction 92.31	Dropped: moment correction fixes the same issue better
Class rows plus mean anchor	Both conditioning signals in one decoder	Ties Ours-A on MNIST/FashionMNIST; SVHN long tail 60.9 vs 65.6 for rows alone	Dropped: the anchor under-disperses samples where means are uninformative

$$\text{Gaussian generator}: \ \tilde{h} \sim \mathcal{N}(m_c, \Sigma_{pooled}) \ \Rightarrow \ \text{Bayes head} = \arg\max_c \ m_c^\top \Sigma^{-1} h - \tfrac{1}{2} m_c^\top \Sigma^{-1} m_c \tag{8}$$

3.5 Moment correction

Moment correction (kept in Ours-A). Moves each fake class onto its exact mean and spread with the smallest change (Proposition 3): +1.6 MNIST, +1.6 FashionMNIST over the plain anchored generator.

Covariance correction: promising, not yet final. Uses each class's exact covariance where the class has enough data: FashionMNIST 81.57 (long tail) and 85.75 (balanced, above the paper's best); unstable on one MNIST seed (Part 8).

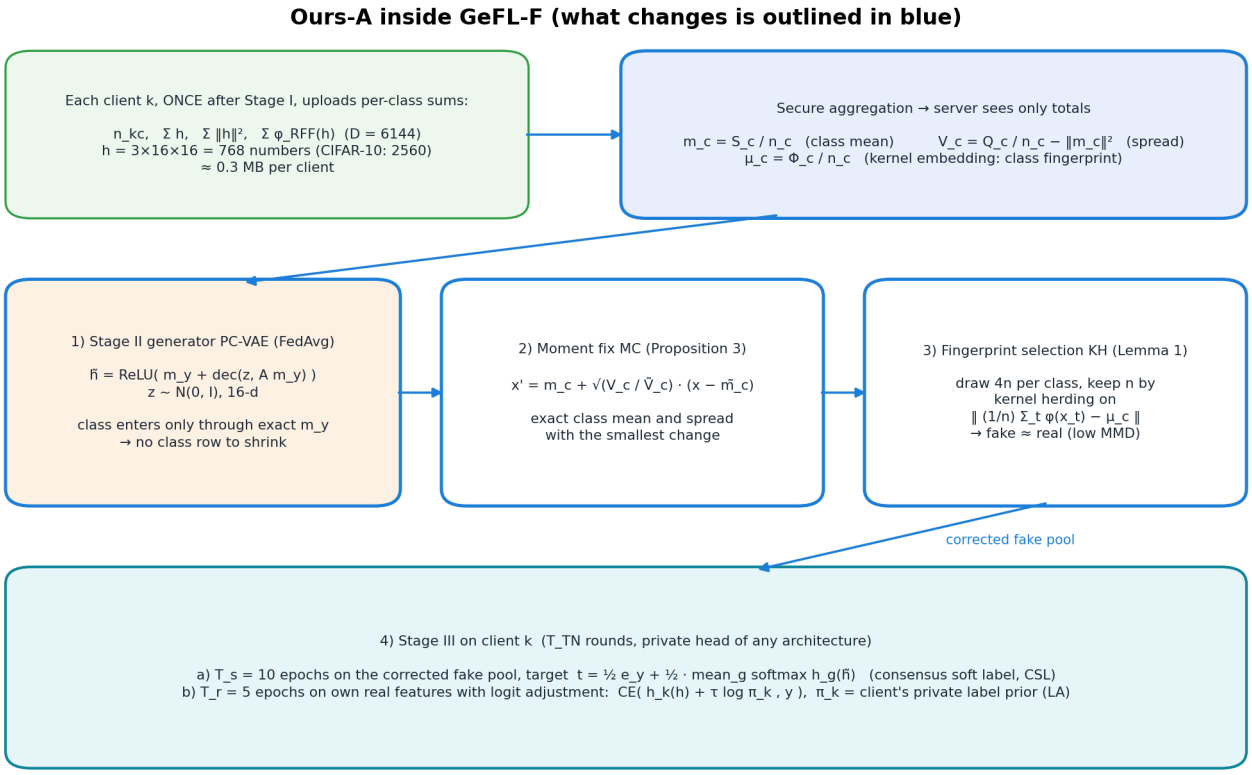
4. The final architecture: Ours-A

The single idea

FedAvg *estimates* class information and, under a long tail, the estimate collapses.

Secure aggregation *adds up* class sums exactly. Ours-A takes class information from these exact sums.

4.1 Ours-A at a glance



Only sums leave the clients; no raw data, features or single-client statistics. Optional (ε, δ)-DP on every release.

Figure 7. Ours-A inside GeFL-F. Blue: what changes.

#	Component	Stage	Runs on	Input → output	Learned?
0	Feature extractor, same as GeFL-F	I	every client	image 1×32×32 → h 3×16×16 = 768	yes, then frozen
1	Class statistics	after I	client → server	own features → 4 sums per class (0.3 MB)	no (exact sums)
2	Mean-anchored generator generator	II	every client, FedAvg	z (16) + m_y (768) → h̃ (768)	yes, then frozen
3	Moment correction: moment fix	before III	server	fake h̃ → fake with exact mean and spread	no
4	Sample selection: fingerprint selection	before III	server	4n candidates → best n per class	no
5	Consensus labels: consensus labels	III	server (sums)	fake h̃ → soft target t (10)	no
6	Logit adjustment: client-prior loss	III	every client	real h, y → loss with offset log π_k	no
7	Private head (10 architectures)	III	every client	h (768) → 10 logits	yes

Rows 1–4 are our contribution (taking class information from exact sums, in three places). Rows 5 and 6 are known techniques (FedMD/FedDF, logit adjustment) that we adopt. Rows 3–6 have no trainable weights.

4.2 Step 1: class statistics (one upload, after Stage I)

Step 1	Class statistics
What	Each client sends, for each of the 10 classes, four sums of its own frozen features.
When	Once, right after Stage I (the feature extractor is frozen, so features never change again).
How	Secure aggregation (Bonawitz et al., CCS 2017): the server receives only the total over all clients.
Why	These totals are exact. Nothing is learned and nothing is averaged, so FedAvg cannot shrink them.

Sum sent by client k for class c	Size	Server computes	Meaning
n_kc: number of samples	1	n_c = Σ_k n_kc	class size
S_kc = Σ h	768	m_c = S_c / n_c	class mean (used by mean-anchored generator and moment correction)
Q_kc = Σ ‖h‖²	1	V_c = Q_c / n_c − ‖m_c‖²	class spread (used by moment correction)
Φ_kc = Σ φ(h), φ = random Fourier features	6,144	μ_c = Φ_c / n_c	class fingerprint (used by sample selection)

$$m_c = \frac{S_c}{n_c}, \quad V_c = \frac{Q_c}{n_c} - \|m_c\|^2, \quad \mu_c = \frac{\Phi_c}{n_c} \tag{9}$$

Total: 10 × (1 + 768 + 1 + 6,144) ≈ 69,000 numbers ≈ **0.3 MB per client, once**. GeFL-F already sends 22 MB per round.

4.3 Step 2: the generator mean-anchored generator (replaces original generator in Stage II)

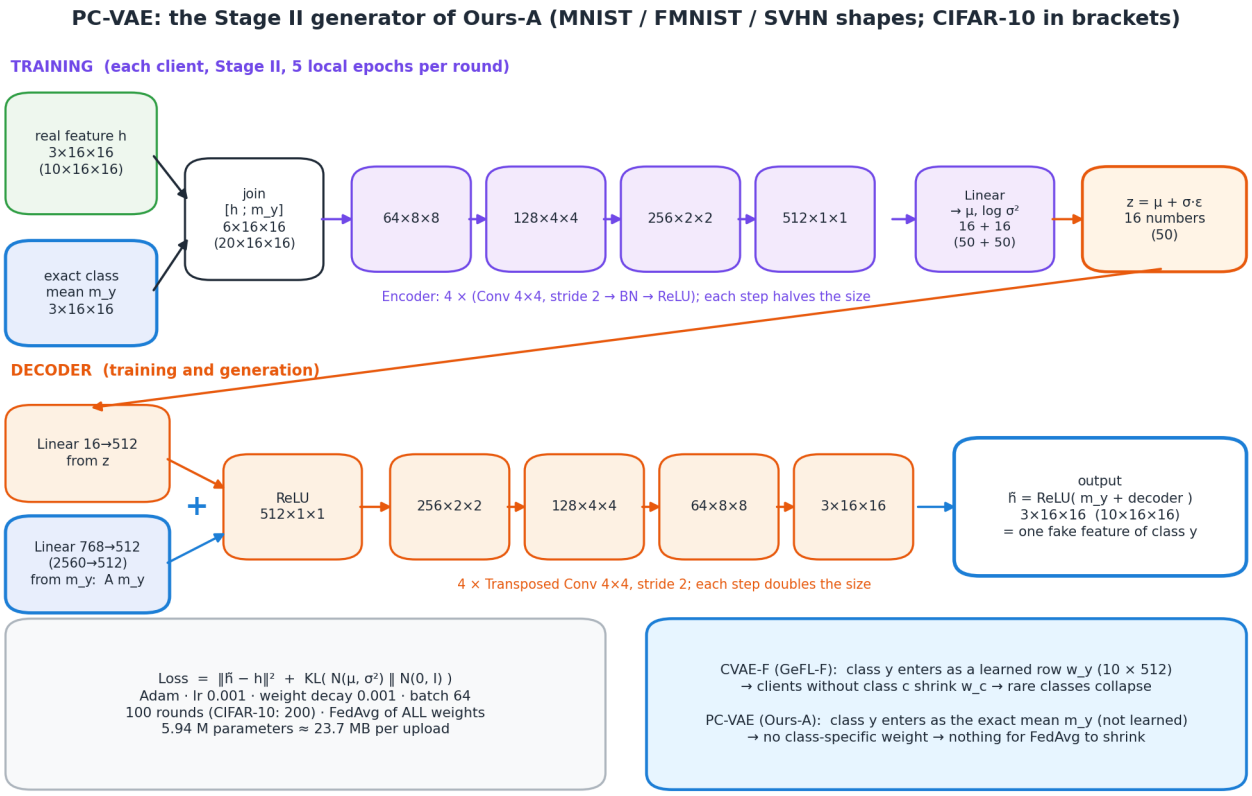


Figure 8. Mean-anchored generator with every shape. Blue boxes: where the exact class mean enters.

Step 2	Mean-anchored generator
Problem it fixes	In original generator the class enters only through a learned row w_y. A client without class c only shrinks w_c (weight decay), so rows of rare classes collapse (Theorem 1, Part 2.2).
What changes	The class enters through the exact mean m_y. It goes in twice: as 3 extra input channels of the encoder, and through a linear layer A (768 → 512) added to the decoder input.
Output	$\tilde{h}$ = ReLU(m_y + decoder output). The network learns only how a sample *differs* from its class mean.
Why it works	There is no class-specific weight, so there is nothing for FedAvg to shrink. The shape of 'difference from the mean' is shared by all classes, so every client trains it, rare classes included.
Source	Class prototypes as shared knowledge: FedProto (Tan et al., AAAI 2022).
Cost	5.94 M parameters (original generator: 5.55 M). Same training as GeFL-F: loss $\|\tilde{h} - h\|^2$ + KL, Adam lr 0.001, wd 0.001, batch 64, 5 local epochs, 100 rounds (CIFAR-10: 200).

$$\tilde{h} = \mathrm{ReLU}(m_y + \mathrm{dec}(z,\ A\,m_y)), \qquad z \sim \mathcal{N}(0, I) \tag{10}$$

Layer	MNIST / FashionMNIST / SVHN	CIFAR-10
Encoder input [h ; m_y]	6×16×16	20×16×16
Conv 4×4 stride 2 → batch norm → ReLU (×4)	64×8×8 → 128×4×4 → 256×2×2 → 512×1×1	same
Linear → μ, log σ²	16 + 16	50 + 50
Decoder input: Linear(z) + A m_y → ReLU	512×1×1	512×1×1
Transposed Conv 4×4 stride 2 (×4)	256×2×2 → 128×4×4 → 64×8×8 → 3×16×16	… → 10×16×16
Output ReLU(m_y + ·)	3×16×16 = 768	10×16×16 = 2560

4.4 Step 3: moment fix

Step 3	Moment correction
Problem it fixes	A variational autoencoder's samples are too narrow (under-dispersed), and the mean of a rare class can be slightly off.
What	For each class, measure the fake mean $\tilde{m}_c$ and spread $\tilde{V}_c$ (from 500 generator samples), then move every fake sample x: $x' = \mathrm{ReLU}(m_c + \sqrt{V_c / \tilde{V}_c} \cdot (x - \tilde{m}_c))$.
Result	The fake class now has the *exact* real mean m_c and spread V_c.
Why this map (Proposition 3)	Of all maps that give the exact mean and spread, this one moves the samples the least. It corrects the class but keeps the shape the generator learned.
Source	Optimal transport between distributions, scalar case (Olkin and Pukelsheim, 1982).
Cost	No training. Two numbers per class (a shift and a scale), computed by the server.

$$T^{\star}(x) = m_c + \sqrt{V_c/\tilde{V}_c}\,(x - \tilde{m}_c) \quad = \quad \arg\min_{T} \; \mathbb{E}\|T(X) - X\|^2 \;\; \text{s.t.} \;\; \mathbb{E}T = m_c,\; \mathbb{E}\|T - m_c\|^2 = V_c \tag{11}$$

4.5 Step 4: fingerprint selection (kernel herding, sample selection)

Step 4	Sample selection
Problem it fixes	Mean and spread can be right while the *shape* is still wrong (e.g. a fake sub-cluster that real data does not have).
What	Draw 4n corrected candidates of class c and keep the n whose average fingerprint is closest to the real fingerprint μ_c.
Why it works (Lemma 1)	A head's error on real data differs from its error on fake data by at most B × (fake-to-real distance). Sample selection directly lowers this fake-to-real distance.
Source	Kernel herding (Chen, Welling and Smola, UAI 2010); maximum mean discrepancy (Gretton et al., JMLR 2012).
Cost	No training. One matrix product per selected sample; uses only μ_c, already uploaded.

Algorithm (greedy, without replacement):

1. compute fingerprints $\varphi(x_i)$ of the 4n candidates (6,144 numbers each); set $w = \mu_c$;
2. pick the candidate with the largest $\langle \varphi(x_i), w \rangle$;
3. update $w \leftarrow w + \mu_c - \varphi(x_{\text{picked}})$; repeat step 2 until n samples are picked.

$$|\mathbb{E}_{G_c}\ell - \mathbb{E}_{P_c}\ell| \;\le\; B \cdot \mathrm{MMD}(G_c, P_c), \qquad x_{t+1} = \arg\max_{x \in \mathcal{X}_c} \langle \phi(x),\, \mu_c \rangle - \frac{1}{t+1} \sum_{s \le t} \langle \phi(x), \phi(x_s) \rangle \tag{12}$$

MNIST, rarest class (24 real samples in the whole federation, held by 8/10 clients): real vs fake, 2-D LDA of real features (grey: all classes)

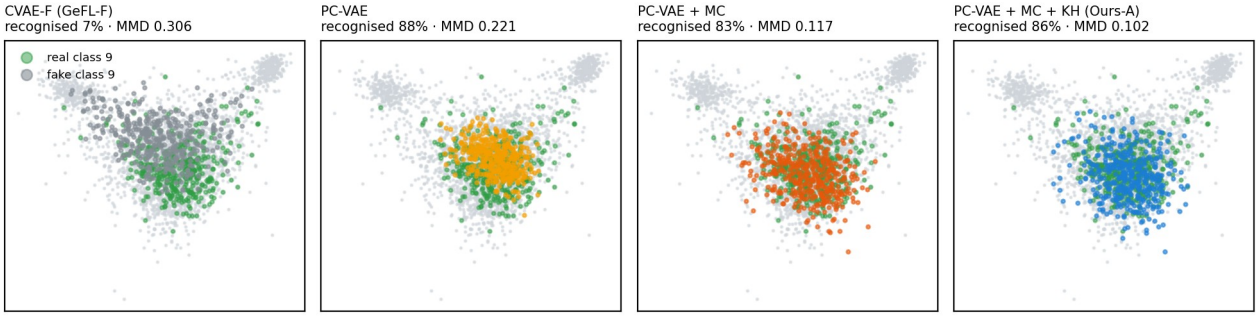


Figure 9. Real data, MNIST's rarest class (24 samples): the generator's output, after moment correction (right centre and width) and after sample selection (right shape). Details in Part 5.9.

4.6 Step 5: training the heads (Stage III)

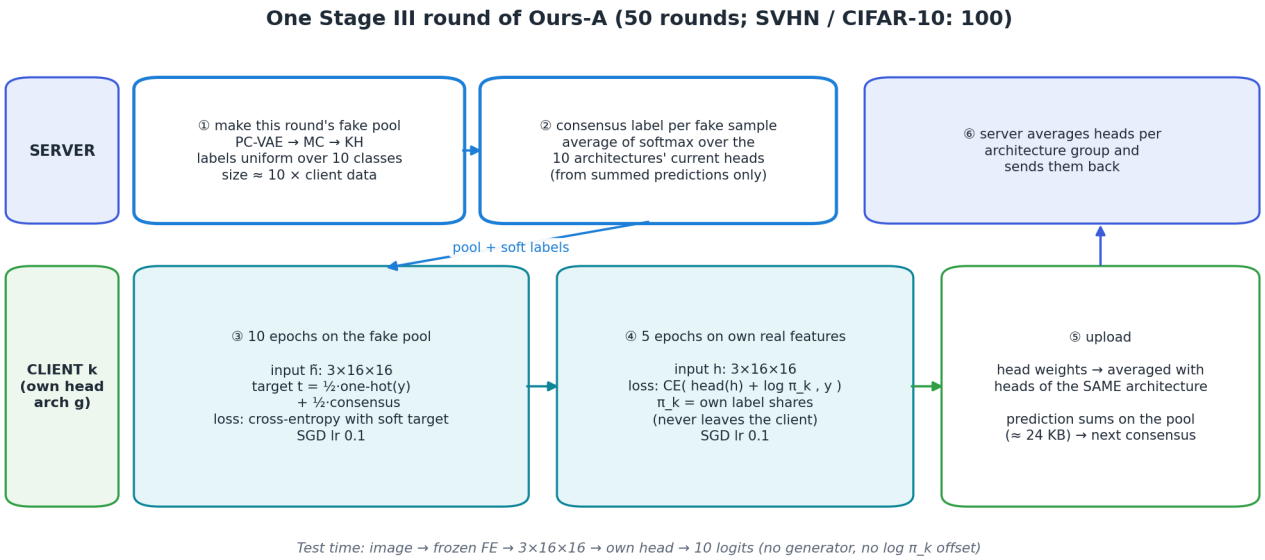


Figure 10. One Stage III round. The fake pool is made fresh every round.

(a) 10 epochs on the fake pool, with consensus soft labels

Step 5a	Consensus labels
What	Target t = ½ · one-hot(y) + ½ · (average of softmax predictions of the 10 architectures' heads on the same fake sample).
How	All clients use one shared fake pool per round. Clients upload only *sums* of predictions (≈ 24 KB per fake epoch), never their heads' outputs on real data.
Why it works	Different architectures make different errors. Their average is a softer and more reliable label than the generator's label alone. β = 0.5 was better than β = 1.
Source	FedMD (Li and Wang, 2019); FedDF (Lin et al., NeurIPS 2020).

$$t(\tilde{h}) = (1-\beta)\, e_y + \beta\, \frac{1}{G} \sum_{g=1}^{G} \mathrm{softmax}\big(h_g(\tilde{h})\big), \qquad \beta = 0.5 \tag{6}$$

(b) 5 epochs on the client's own real features, with logit adjustment

Step 5b	Logit adjustment
What	Loss = cross-entropy(head(h) + τ · log π_k , y), τ = 1, where π_k is the client's own label distribution.
Why it works	A client with 90 % of one class learns to predict that class. Adding log π_k during training removes this bias; at test time the offset is dropped.
Privacy	π_k never leaves the client.
Source	Logit adjustment (Menon et al., ICLR 2021); FedLC (Zhang et al., ICML 2022).

$$\ell_{LA}(h, y) = -\log \frac{\exp(f_y(h) + \tau \log \pi_{k,y})}{\sum_j \exp(f_j(h) + \tau \log \pi_{k,j})} \tag{5}$$

(c) Why 10 fake epochs instead of GeFL-F's 1

Proposition 5: more fake training helps only when the generator is faithful. Measured on MNIST (long tail): +1.7 with mean-anchored generator + moment correction + sample selection, +3.9 at 100 clients, but −0.05 with the original generator.

(d) Aggregation and test time

- Heads are averaged only with heads of the **same architecture**, as in GeFL-F.
- Test: image → frozen feature extractor → 3×16×16 → the client's own head → 10 logits. No generator is needed.

4.7 How each step adds up (same seeds, long tail, balanced accuracy %)

Step	MNIST	FashionMNIST	What it fixes
GeFL-F	75.25	58.47	—
+ logit adjustment	78.92	64.78	client's own class skew
+ holder-weighted averaging (class rows averaged over holders)	87.89	76.02	rare rows no longer pulled down
+ consensus labels	90.40	76.60	better labels on fake features
Mean-anchored generator replaces holder-weighted averaging	90.75	75.96	no class rows at all
+ moment correction	92.31	77.56	exact mean and spread
+ sample selection	93.11	78.22	fake ≈ real (low fake-to-real distance)
+ 10 synthetic epochs = **Ours-A**	**94.82**	**79.67**	uses the now-faithful generator more

4.8 Where each part comes from

Part of Ours-A	Source (research paper)	What we use from it
Class sums sent safely	Bonawitz et al., Practical Secure Aggregation, ACM CCS 2017	server gets only the total over clients
Class mean as class identity	Tan et al., FedProto, AAAI 2022	class prototypes (means) as shared knowledge
Moment fix	Optimal transport, scalar case (Olkin and Pukelsheim 1982)	smallest change that gives the exact mean and spread
Fingerprint selection	Chen, Welling and Smola, Kernel Herding, UAI 2010; Gretton et al., JMLR 2012	pick samples whose average fingerprint matches the class
Client-prior loss	Menon et al., Logit Adjustment, ICLR 2021; Zhang et al., FedLC, ICML 2022	loss that undoes a client's own class skew
Consensus labels	Li and Wang, FedMD, 2019; Lin et al., FedDF, NeurIPS 2020	average prediction of different models as soft label

What is new in Ours-A: taking class information from exact secure-aggregation sums instead of from FedAvg-trained weights, and using it in three places (generator input, moment fix, sample selection). Each building block exists; the combination, its use inside a heterogeneous generator-based pipeline, and the collapse analysis that motivates it are ours. Logit adjustment, consensus labels and holder-weighted averaging are adopted, not claimed.

4.9 Communication and privacy

Release	Size	When	Protection
Class counts, sums of h and of $\|h\|^2$, sums of random features	≈ 0.3 MB per client	once, after Stage I	secure aggregation; optional (ε, δ) differential privacy
Consensus labels: prediction sums on a shared fake pool	≈ 24 KB per fake epoch	each Stage-III round	sums only; no real data
Generator	5.94 M params ≈ 23.7 MB (GeFL-F: 22 MB)	each Stage-II round	same as GeFL-F
Feature extractor, heads	as in GeFL-F	as in GeFL-F	unchanged

For formal privacy, features are clipped and each release gets Gaussian noise (zero-concentrated differential privacy):

$$\sigma_{rel} = \sqrt{2/\rho}, \qquad \rho = \left(\sqrt{\log(1/\delta) + \varepsilon} - \sqrt{\log(1/\delta)}\right)^2 \quad \text{(zCDP, split over 4 releases)} \tag{13}$$

5. Results

5.1 Protocol

All runs follow the paper's appendix: the same models, optimisers, rounds and data fractions; 3 seeds; each seed fixes the split and the initialisation for every method, so comparisons are paired; p-values come from paired t-tests. Local runs used one laptop GPU; heavy runs used Kaggle T4 × 2 sessions.

5.2 Main result: long-tailed clients

Method (imbalance 100 : 1, Dirichlet 0.5, 10 clients; balanced accuracy)	MNIST	FashionMNIST	CIFAR-10	SVHN
GeFL-F (Kang et al., 2025)	75.25	58.47	33.74	50.49
GeFL-F + logit adjustment (similar to FedLC)	78.92	64.78	37.30	56.44
+ holder-weighted averaging + logit adjustment	87.89	76.02	40.73	62.45
+ holder-weighted averaging + logit adjustment + consensus labels	90.40	76.60	42.10	–
Gaussian generator from exact statistics + logit adjustment	87.04	73.60	44.17	44.23
GeFL-F + last-layer refit on pooled real data (upper bound of CCVR / CReFF / Fed3R-type recalibration)	90.7	79.2	–	64.8
Ours-A	**94.82**	**79.67**	**44.38**	54.41
Ours-A with the SVHN switch (class rows + holder-weighted averaging + moment correction + sample selection when class means are uninformative)	94.82	79.67	44.38	**65.60**
oracle: Ours-A's heads, last layer re-fit on pooled real data	94.93	81.55	50.64	59.13 / 68.4 (switch)

The red row is the strongest baseline we found, and it is new in this version: it shows how much of GeFL-F's loss a known classifier recalibration would recover. Ours-A beats it by +4.1 (MNIST), +0.5 (FashionMNIST) and +0.8 (SVHN with the switch). On FashionMNIST the margin is within noise; on CIFAR-10 the number is not yet logged (Part 8).

Tail recall: 51.9 → 91.1 (MNIST), 35.4 → 73.6 (FashionMNIST). SVHN is the weak case (Part 5.7).

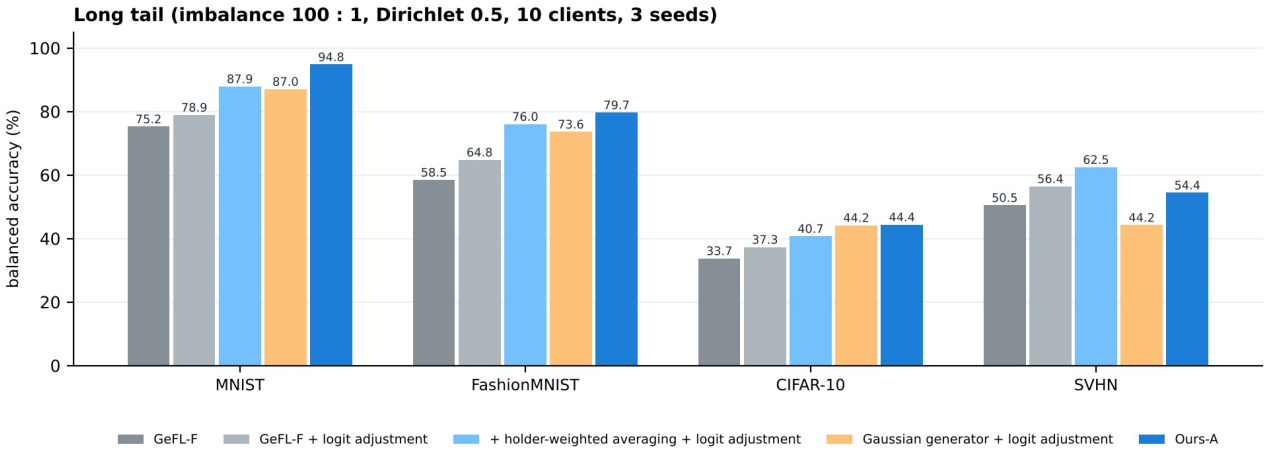


Figure 11. Long-tailed clients: Ours-A against GeFL-F and the strongest alternatives.

5.3 The paper's own balanced setting, against all ten of its methods

best_mean_acc, balanced	GeFL-F (paper)	Best of the paper's ten methods	Our GeFL-F	Ours-A	Ours-A − best
MNIST, 10 clients	95.47	96.44 (GeFL, diffusion)	96.01	**97.62**	+1.18
MNIST, 50 clients	95.04	95.04 (GeFL-F, original generator)	95.43	**96.93**	+1.89
MNIST, 100 clients	94.63	94.63 (GeFL-F, original generator)	94.88	**96.77**	+2.14
FashionMNIST, 10 clients	83.14	84.28 (GeFL-F, diffusion generator)	82.76	83.96	−0.32
FashionMNIST, 50 clients	82.21	82.96 (GeFL-F, diffusion generator)	81.50	**82.96**	±0.00
CIFAR-10, 10 clients	55.86	59.36 (GeFL, diffusion)	59.10	**60.93**	+1.57
SVHN, 10 clients (mean anchor)	76.26	76.26 (GeFL-F, original generator)	75.82	74.38	−1.88
SVHN, 10 clients (with the switch to class rows)	76.26	76.26 (GeFL-F, original generator)	75.82	**77.07**	+0.81
FashionMNIST, 10 clients, with covariance correction (covariance version)	83.14	84.28 (GeFL-F, diffusion generator)	82.76	**85.75**	+1.47

Above the best of the paper's 10 methods on MNIST (all client counts), CIFAR-10 and, with the switch, SVHN; level on FashionMNIST with the main method and above it with covariance correction. Note: best_mean_acc picks each architecture's best round on the test set, which flatters every method equally; our long-tail results use the final round instead.

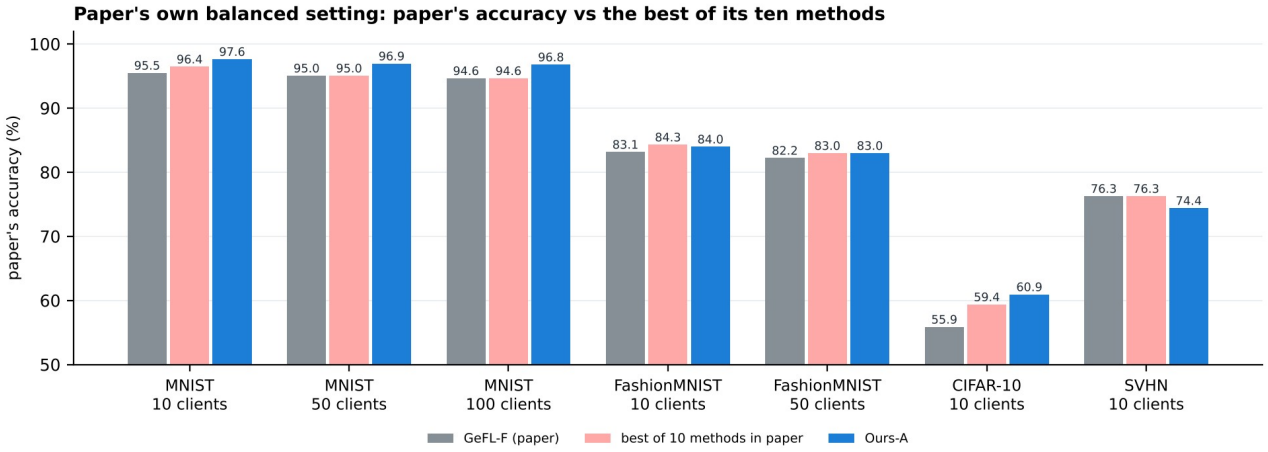


Figure 12. The paper's balanced setting: Ours-A against the paper's GeFL-F and the best of its ten methods.

5.4 Data augmentation (paper's Table IV), and why MixUp does not transfer

CIFAR-10, balanced, 10 clients	Accuracy
FedAvg / + MixUp / + CutMix / + AugMix / + AutoAugment (paper)	55.65 / 60.07 / 58.95 / 53.96 / 56.99
Image-space GeFL + MixUp / + CutMix / + AugMix / + AutoAugment (paper)	62.67 / 61.66 / 56.47 / 59.97
Ours-A, no augmentation	60.93
Ours-A + feature-space MixUp (our run, same seeds)	59.98
Ours-A + our mixing variant	60.29

Ours-A beats 7 of 9 without augmentation. The two higher ones train full models on raw images; in GeFL-F the extractor is frozen, and feature-space MixUp lowered our accuracy (59.98 vs 60.93).

5.5 More clients, and formal privacy

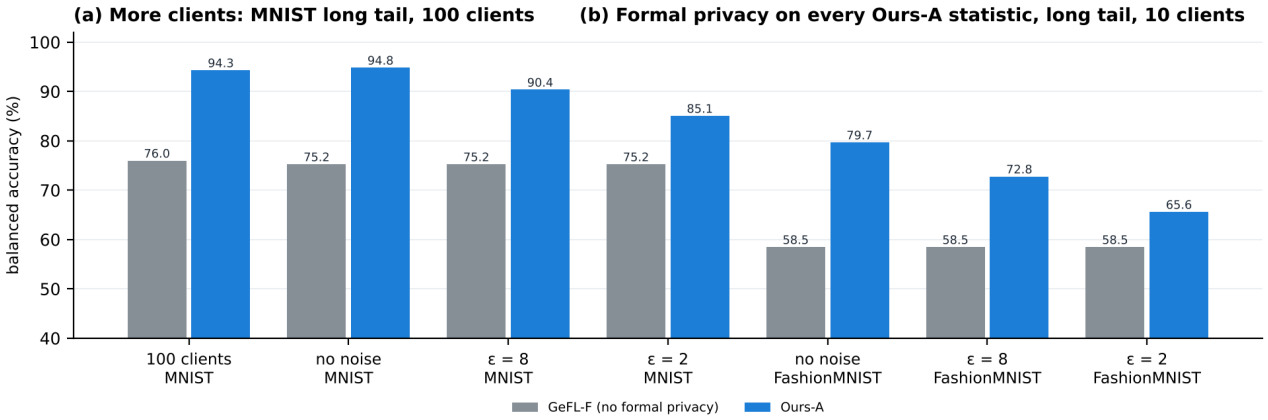


Figure 13. (a) 100 clients, MNIST long tail. (b) Ours-A with every statistic made differentially private (ε, δ = 10^{-5}).

Setting	GeFL-F	Holder-weighted averaging + logit adjustment	Gaussian generator + logit adjustment	Ours-A
MNIST long tail, 100 clients	75.98	83.07	86.05	**94.30**

Only −0.5 from 10 to 100 clients (the original generator with holder-weighted averaging loses 4.8): no class weights to estimate from shrinking client data. FashionMNIST long tail (3 seeds): Ours-A 79.7 / 79.5 / 79.0 at 10 / 50 / 100 clients, against GeFL-F 58.5 / 61.9 / 61.9.

Long tail, 10 clients	GeFL-F (no formal guarantee)	Ours-A exact	Ours-A, ε = 8	Ours-A, ε = 2
MNIST	75.25	94.82	90.36	85.06
FashionMNIST	58.47	79.67	72.77	65.59

Privacy costs 4.5–6.9 points at ε = 8 and 9.8–14.1 at ε = 2, mostly on the rarest classes; still +7 to +15 over GeFL-F.

5.6 What each component contributes

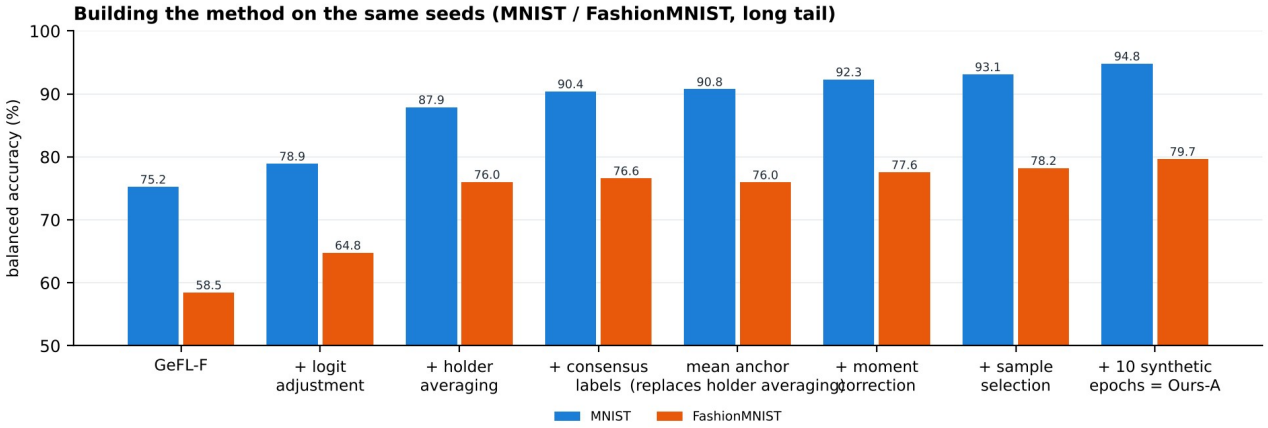


Figure 14. Adding components one at a time on the same seeds (long tail).

Each step adds on the same seeds. Kernel herding helped in all 9 paired tests (p = 0.004).

Leave-one-out ablation (K09: remove one part from the full Ours-A; long tail, 3 seeds, paired)

Removed	MNIST (change, points)	FashionMNIST (change, points)	Reading
Mean-anchored generator (back to original generator + holder-weighted averaging)	−4.72 (p = 0.02)	−0.67	large on MNIST, small on FashionMNIST
Logit adjustment	−2.98 (p = 0.02)	−6.18 (p = 0.02)	largest; mostly recovered by post-hoc bias calibration
Consensus labels	−1.63	−1.38 (p = 0.008)	medium
Moment correction	−0.96 (p = 0.03)	−0.49	small; works through the spread
Sample selection	−0.71	−0.30	small

Full Ours-A in this run: 95.18 ± 0.08 (MNIST), 79.79 ± 0.31 (FashionMNIST). All 30 seed-level differences are negative (every part, every seed, both datasets; sign test $p = 2 \cdot 10^{-9}$). With 3 seeds no single part survives a multiple-test correction; two more seeds (job K14, ready) are needed for per-part significance.

5.7 The weak case: SVHN

SVHN	GeFL-F	+ holder-weighted averaging + logit adjustment	Ours-A
Long tail, balanced accuracy	50.49	62.45	54.41
Balanced setting, paper's accuracy (best_mean_acc)	75.82	75.64	74.38

SVHN's class means are almost identical (nearest-mean accuracy 12 %, chance 10 %), so a mean-anchored generator cannot separate its classes.

Switch (implemented): the server measures how well the exact class means separate the classes (F, from the same sums). When F < 0.05 it keeps GeFL-F's class rows, protected by holder-weighted averaging, and still applies moment correction and sample selection. On SVHN this gives **65.60 ± 1.50** under the long tail (+15.1 over GeFL-F) and **77.07 ± 0.19** in the balanced setting (above the paper's 76.26). MNIST, FashionMNIST and CIFAR-10 are unaffected (F = 0.07–0.60).

5.8 Mechanism measurements

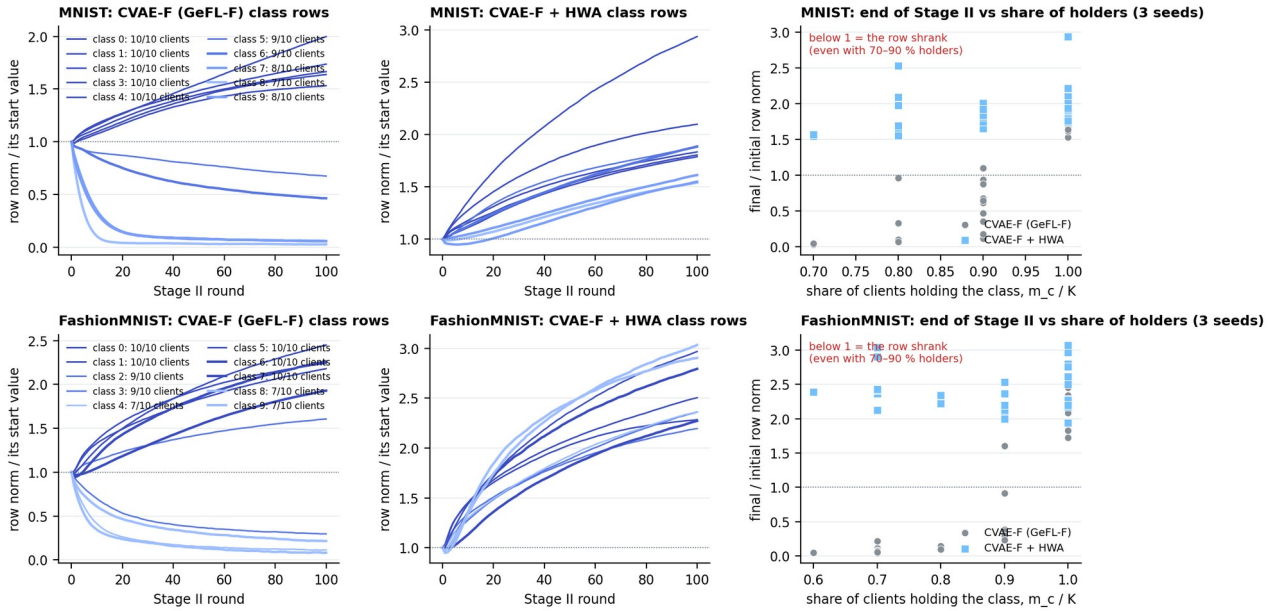
Quantity (long tail)	GeFL-F	Ours-A
Tail fidelity (referee accuracy on synthetic tail features), MNIST / FashionMNIST	10.3 / 19.9 %	88.6 / 78.7 %
Tail spread relative to real data, raw → after correction and herding	–	0.39–0.49 → 0.93–0.98
Fake-sample correctness in the balanced setting vs the judge's accuracy on real data (FashionMNIST)	–	0.837 vs 0.839

After the fixes, fake features are as hard as real ones, not artificially clean.

5.9 Proof on real data: what each component does

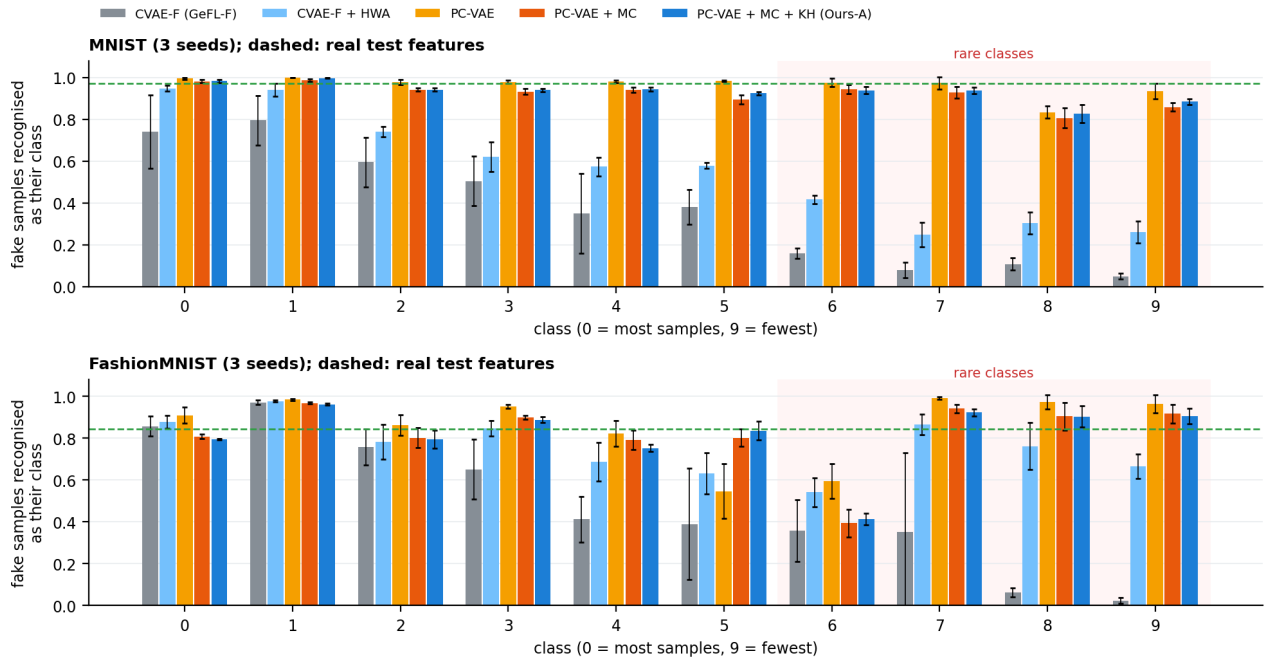
Measured on real features: MNIST and FashionMNIST, long tail, 3 seeds, the same splits as every run (Kaggle T4; a second run on another GPU gives the same numbers within about 1 point). The *referee* is a head trained on held-out real features; it never takes part in training. Code: experiments/D01_component_evidence.py.

Claim: under FedAvg the learned class rows of rare classes shrink every round; HWA stops it



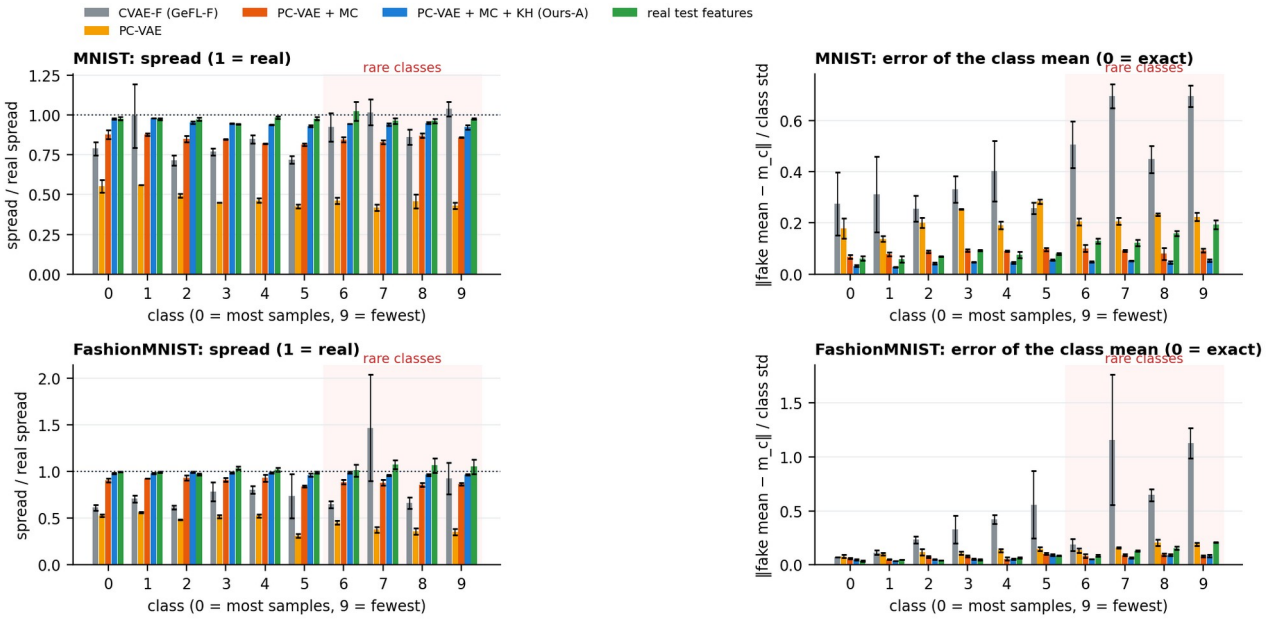
Rows of rare classes under FedAvg end at 0.33 (MNIST) and 0.44 (FashionMNIST) of their start value; with holder-weighted averaging they grow (×1.7, ×2.4). They shrink even when 70–90 % of clients hold the class.

Claim: GeFL-F's fake features of rare classes do not look like their class; the exact-mean anchor fixes it (referee = a head trained on held-out REAL features; it labels 1,000 fake samples per class)



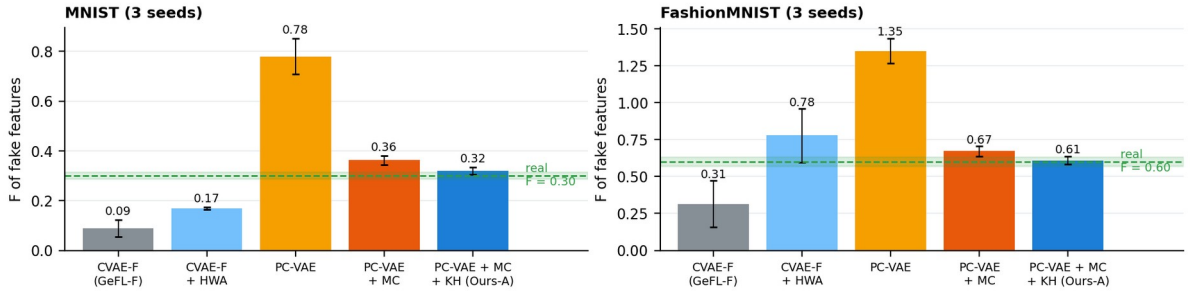
Rare-class fake samples recognised as their class: GeFL-F 10 % / 20 %, + holder-weighted averaging 31 % / 71 %, mean-anchored generator 93 % / 88 %, Ours-A 90 % / 79 % (real test features: 97 % / 84 %).

Claim: PC-VAE's samples are too narrow (spread ≈ 0.5 of real); MC fixes the mean and most of the spread (ReLU after the map cuts a little), KH brings the spread closest to real



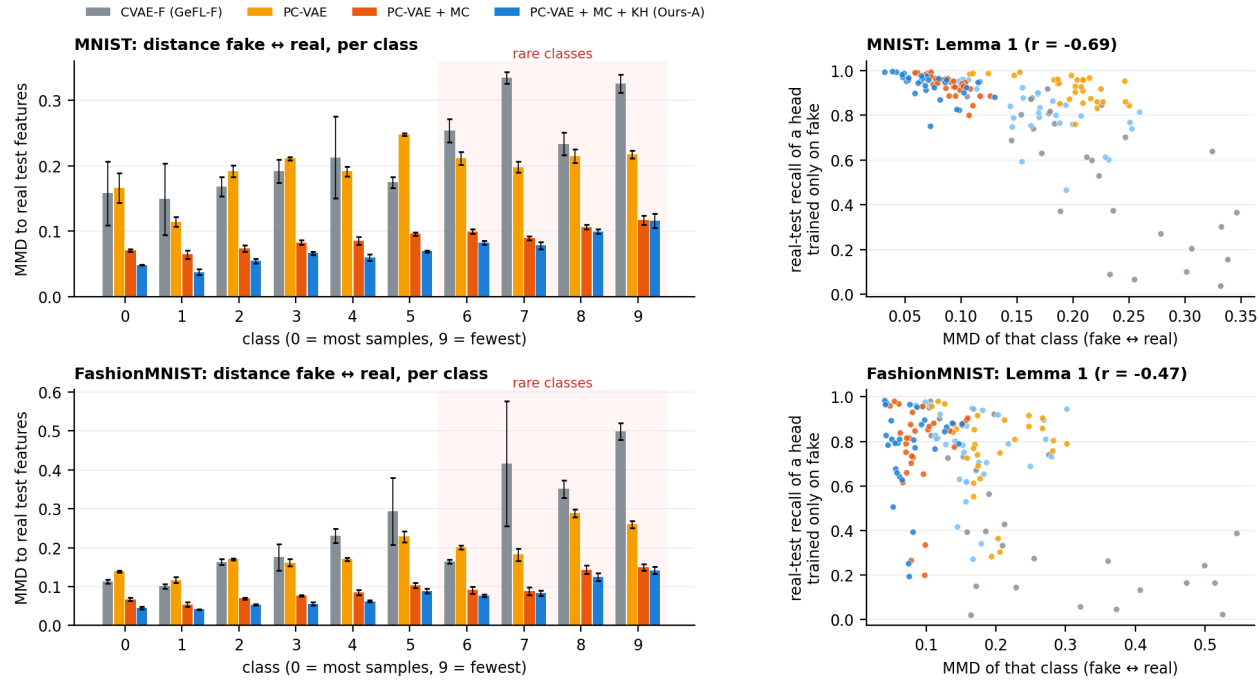
Spread of fake features (1 = real): mean-anchored generator 0.47 / 0.45, + moment correction 0.85 / 0.90, + sample selection 0.95 / 0.98. Rare-class mean error: 0.22 → 0.09 → 0.05 class std (MNIST).

Claim: GeFL-F's fake classes overlap (F too low); PC-VAE's are too far apart (too clean, F too high); MC + KH make them about as separable as real classes (F = spread of class means / spread within a class)



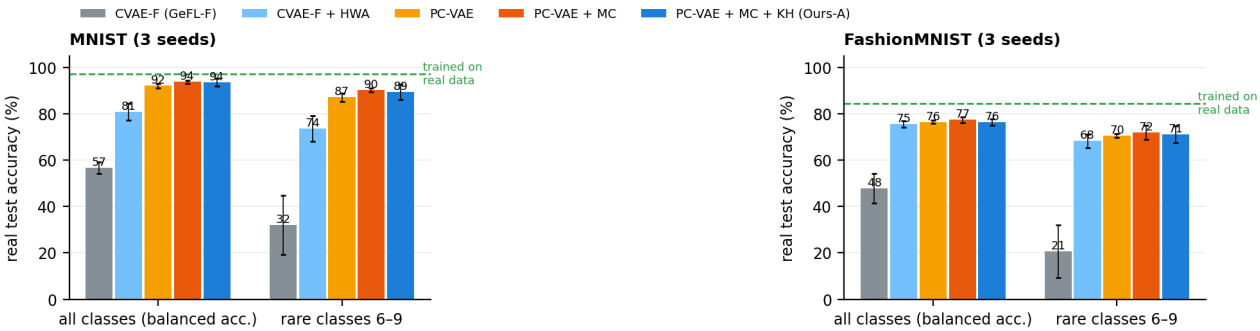
Class separability F of fake features: GeFL-F 0.09 / 0.32 (classes overlap), mean-anchored generator 0.78 / 1.35 (too clean), Ours-A 0.32 / 0.61, real 0.30 / 0.60.

Claim: KH brings fake features closest to real ones (lowest MMD), and lower MMD means better real accuracy



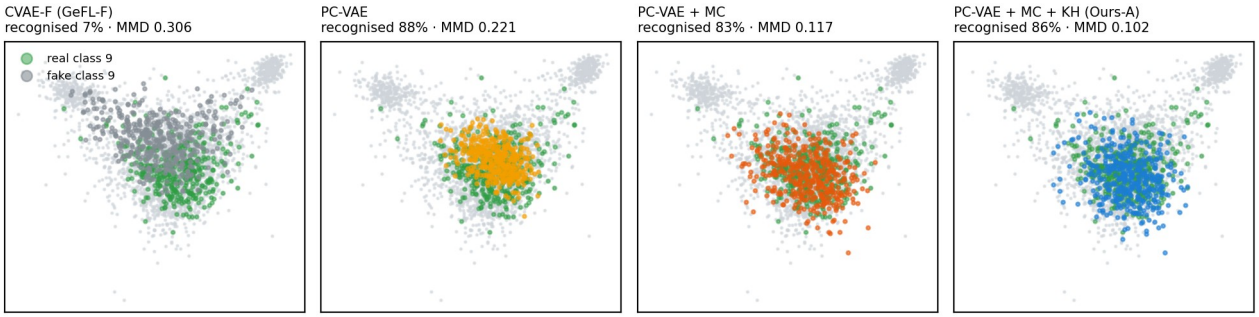
Distance to real test features (fake-to-real distance, all classes): GeFL-F 0.22 / 0.25 → mean-anchored generator 0.20 / 0.19 → + moment correction 0.089 / 0.093 → + sample selection 0.071 / 0.078. Lower fake-to-real distance goes with higher real accuracy (r = −0.69 / −0.47), as Lemma 1 says.

Claim: fake features of Ours-A teach real classes. A head trained ONLY on 1,000 fake features per class, tested on real data

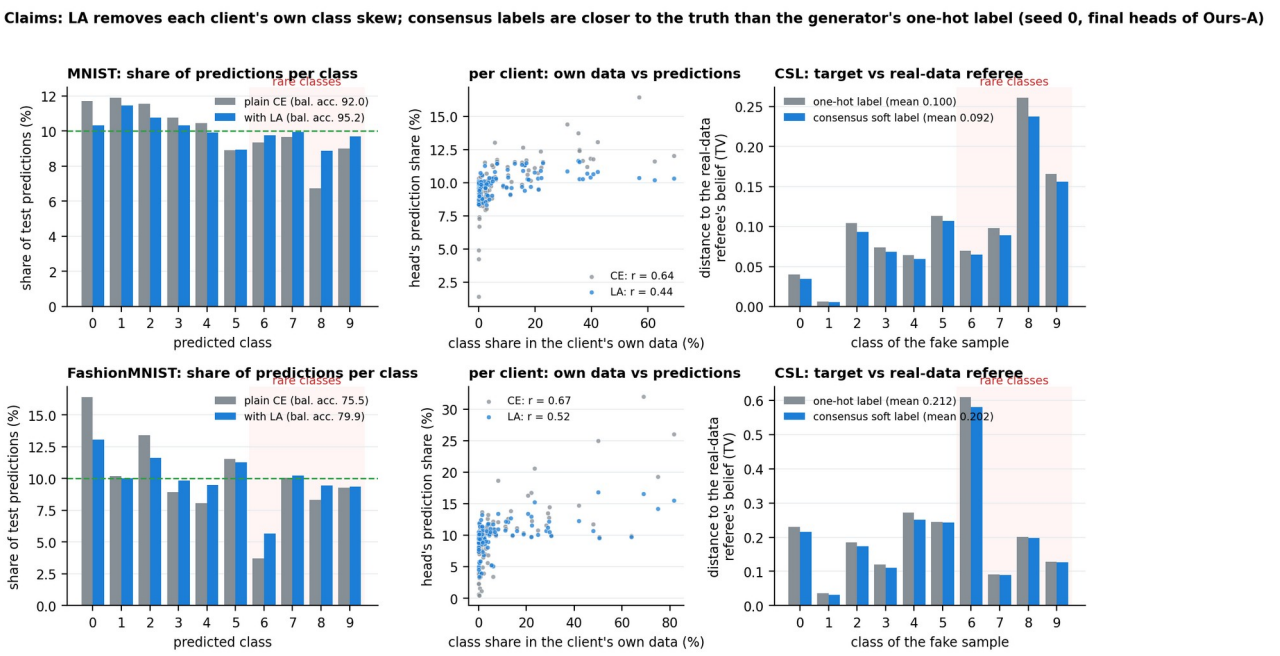


Head trained only on fake features, tested on real data (balanced accuracy): GeFL-F 57 / 48, + holder-weighted averaging 81 / 76, mean-anchored generator 92 / 76, + moment correction 94 / 77, + sample selection 94 / 76. Rare classes on MNIST: 32 → 89. Sample selection does not add in this test (−0.2 / −1.1); its gain shows only inside the full pipeline (+0.8 / +0.7).

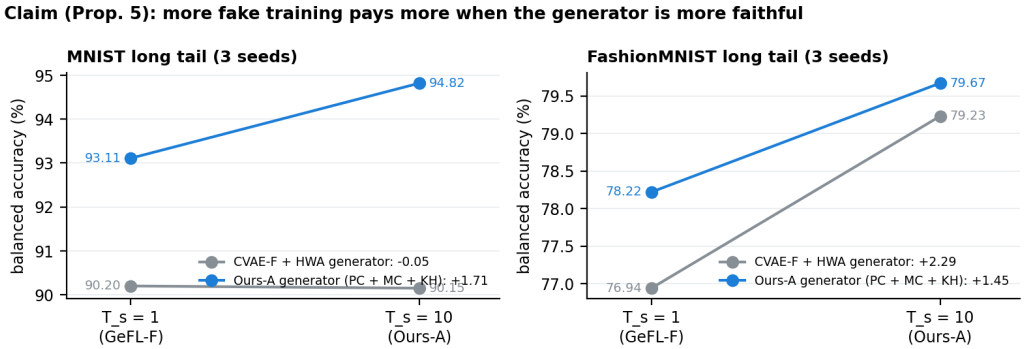
MNIST, rarest class (24 real samples in the whole federation, held by 8/10 clients): real vs fake, 2-D LDA of real features (grey: all classes)



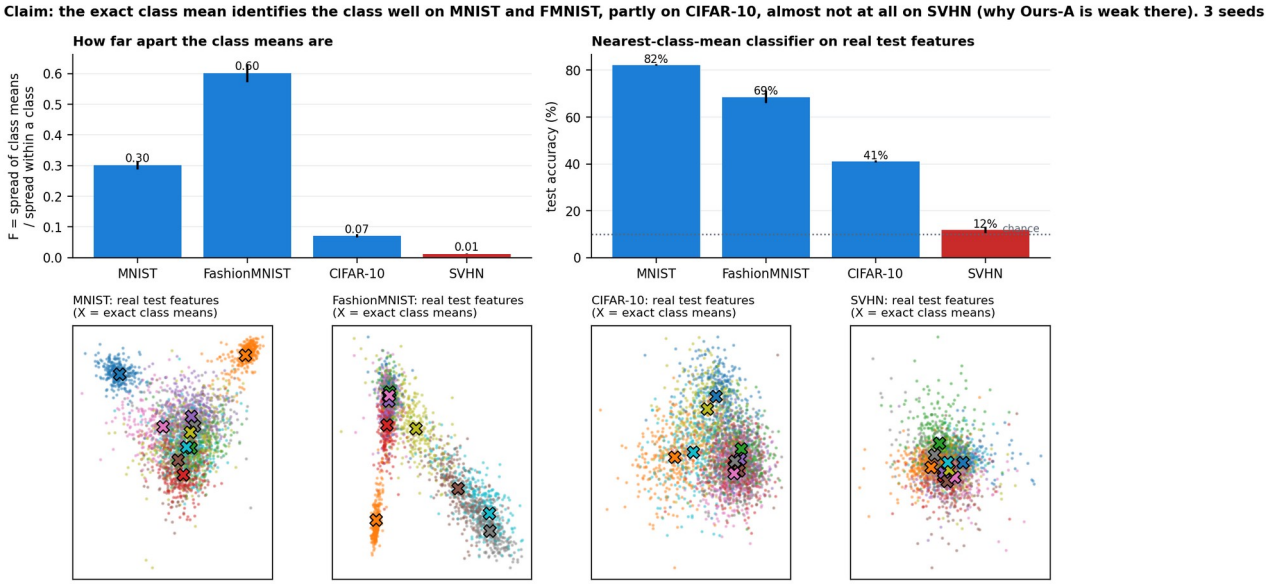
MNIST's rarest class (24 real samples): GeFL-F's fake class 9 lies on other classes; mean-anchored generator is on the class but too narrow; moment correction and sample selection spread it over the real class.



Logit adjustment (seed 0): balanced accuracy 92.0 → 95.2 (MNIST), 75.5 → 79.9 (FashionMNIST); correlation between a client's own class shares and its predictions 0.64 → 0.44, 0.67 → 0.52. Consensus labels: the soft target is closer to the real-data referee than the one-hot label, lower or equal in every class (total-variation distance 0.100 → 0.092, 0.212 → 0.202).



More fake training helps more with the better generator (MNIST: −0.05 vs +1.71).



How well the exact class mean identifies the class (3 seeds): F = 0.30 / 0.60 / 0.07 / 0.012 and nearest-mean accuracy 82 / 69 / 41 / 12 % on MNIST / FashionMNIST / CIFAR-10 / SVHN (chance 10 %). SVHN's means carry almost no class information: the reason Ours-A is weak there.

6. Where the remaining error lives: the last layer

Every run also logs two diagnostics. **Last-layer refit (oracle):** keep each head's body, re-fit only its last layer on the pooled real training features with balanced sampling. **Ensemble:** average the 10 heads' predictions. The difference *refit − head* is the error in the last layer alone (the *readout gap*).

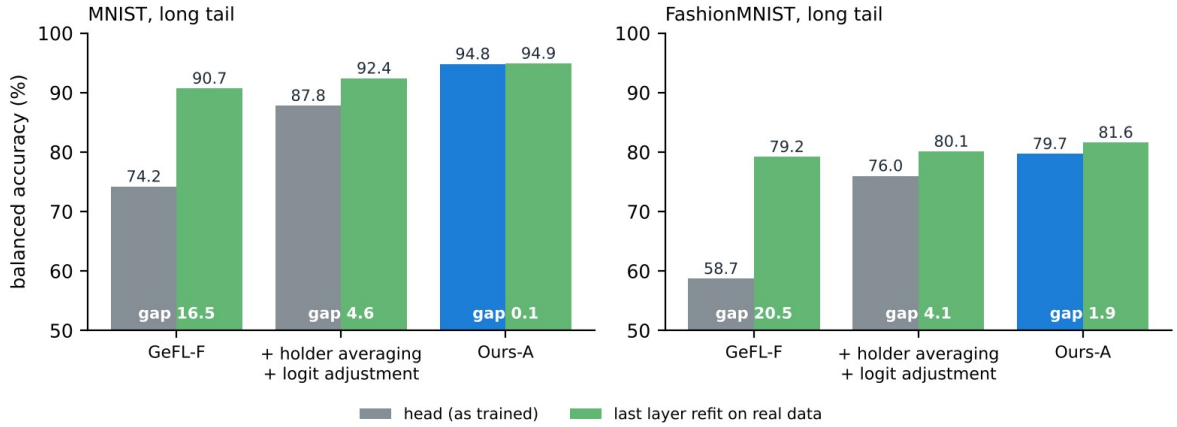


Figure 15. Head as trained vs the same head with its last layer re-fit on real data (long tail, 3 seeds).

Run (long tail, 3 seeds)	Head	Last layer refit	Readout gap
GeFL-F, MNIST / FashionMNIST / SVHN	74.2 / 58.7 / 50.5	90.7 / 79.2 / 64.8	**16.5 / 20.5 / 14.3**
+ holder-weighted averaging + logit adjustment, MNIST / FashionMNIST	87.8 / 76.0	92.4 / 80.1	4.6 / 4.1
Ours-A, MNIST / FashionMNIST	94.8 / 79.7	94.9 / 81.6	0.1 / 1.9
Ours-A, FashionMNIST at 10 / 50 / 100 clients	79.7 / 79.5 / 79.0	81.6 / 81.2 / 81.0	1.9 / 1.7 / 2.0
Ours-A (switch), SVHN; Ours-A, CIFAR-10	65.6; 46.1	68.4; 52.3	2.8; **6.2**

- **GeFL-F's long-tail failure is mostly in the last layer.** Its bodies are good; a last-layer refit adds 14–20 points.
- **Ours-A removes most of it** (16.5 → 0.1 on MNIST) *and* gives better bodies: Ours-A's refit beats GeFL-F's refit by +4.2 / +2.4 / +3.6.
- **What is left does not shrink with more clients** (1.7–2.0 on FashionMNIST at every K) and is largest on CIFAR-10 (6.2).

6.1 A federated way to close the gap: exact readout

The refit above needs pooled real data, which federated learning forbids. But the same answer can be built from sums. For each architecture g, every client passes its real features through head g's body and sends, by secure aggregation, per-class sums of the embeddings e, the class counts and the sum of $e \cdot e^T$. The server then sets the last layer in closed form (linear discriminant analysis with a uniform class prior):

- class means μ_c and shared covariance Σ from the sums (shrunk: $0.9 \cdot \Sigma + 0.1 \cdot \mathrm{tr}(\Sigma)/d \cdot I$, fixed in advance);
- weights $w_c = \Sigma^{-1} \mu_c$, bias $b_c = -\frac{1}{2} \mu_c^T \Sigma^{-1} \mu_c$.

Cost: one extra round; under 1.1 M numbers per architecture for the largest head. It discloses federation-level class statistics of the embeddings, the same kind of release as the class sums of Part 4.2.

Test (T03, pre-registered; 1 seed, shortened schedule on CPU)

Run	Head	Refit (pooled data)	Exact readout (sums only)	Bias calibration (offsets only)
FashionMNIST GeFL-F	61.2	78.7	**79.1**	65.1

Run	Head	Refit (pooled data)	Exact readout (sums only)	Bias calibration (offsets only)
FashionMNIST + holder-weighted averaging + logit adjustment	73.0	79.4	**79.9**	73.6
FashionMNIST Ours-A	75.7	80.7	**80.3**	77.9
MNIST GeFL-F	72.4	91.0	**90.8**	75.5
MNIST + holder-weighted averaging + logit adjustment	82.1	91.8	**91.3**	82.0
MNIST Ours-A	91.1	93.6	**92.8**	92.2

- **Passed:** exact readout is within 0.8 points of the pooled-data refit in all runs (9 of 9 including the follow-up), +18 points on plain GeFL-F. Logit offsets (bias calibration, logit adjustment) stay below the refit in every run: the error is in the weights, not only in a class prior.
- **Where the error sits (measured):** on classes a client does **not hold** (FashionMNIST Ours-A: −17 recall points against the refit), less on classes held with 1–20 samples (−7), none on well-held classes.
- **Failed prediction (reported honestly):** we predicted the cause was a wrong synthetic class mean; that did not explain the per-class gap. Mis-scaled weight lengths explain about a quarter (re-normalising them recovers 16–34 % of the gap). The full mechanism is still open.

6.2 What this means for the paper

- **Exact readout is not a new idea**: it is the known classifier recalibration from federated statistics (CCVR, NeurIPS 2021; CReFF, IJCAI 2022; Fed3R, ICML 2024), extended to heterogeneous heads (one set of statistics per architecture). We present it that way.
- **GeFL-F + recalibration must be a baseline.** Against it Ours-A wins by +4.1 / +0.5 / +0.8, not +19.6 / +21.2 / +15.1. With recalibration applied to both, Ours-A keeps +4.2 / +2.4 / +3.6.
- Exact readout is now logged automatically (er_bal) in every new run, at no extra GPU cost.

7. Why it fails, and why each fix works

Parts 4 and 5 say what each part of Ours-A does and how many points it adds. This part says **why**. There are eight failures, in the order they occur. For each one we give: what goes wrong; the cause, step by step; the measurement that shows it; what we would have measured *if the explanation were wrong*; why the fix removes the cause; and how sure we are.

#	What goes wrong	Cause in one line	Fix	Before → after
1	Rare classes are erased from the generator	Clients without the class pull its list to zero every step; clients with it push back only rarely	Holder-weighted averaging, then the mean-anchored generator	Rare lists 4–10 % → 117–125 % of start; rare fakes recognised 10 % → 93 % (MNIST)
2	Fakes too narrow and too clean	The generator learns to output the "typical" sample	Moment correction	Spread 0.47 → 0.85 of real; separability 0.78 → 0.32 (real 0.30)
3	Fakes have the wrong shape	Average and spread are only two numbers	Sample selection	Fake-to-real distance 0.089 → 0.071
4	Each model leans to its client's common classes	The last 5 epochs of every round are on skewed real data	Logit adjustment	Skew 0.64 → 0.44; +3 to +6 points
5	Ambiguous fakes get a confident label	A fake drawn as class c can look like two classes	Consensus labels	Closer to a real-data judge in every class; +1.4 to +1.6 points
6	More fake training did not help	Training longer on wrong fakes teaches the wrong thing longer	Only after the generator is fixed	−0.05 → +1.7 points
7	Ours-A weak on SVHN	SVHN's class averages are nearly identical	Automatic switch back to class lists	54.4 → 65.6
8	Some error stays in the last layer	Weights of classes the client lacks have the wrong size (about a quarter of it); the rest unknown	Exact readout from sums	Within 0.8 points of the best possible last layer

7.1 Rare classes are erased from the generator

What goes wrong. The original generator forgets how to draw rare classes. Its fake samples of a rare class look like random classes, so every model trained on them learns nothing about that class, and rare-class accuracy stays at 52 % (MNIST) and 35 % (FashionMNIST).

The cause, step by step.

- **The generator keeps one list of 512 numbers per class** (its *class list*). To draw a sample of class c it reads list c. A long, distinctive list says "class c" clearly; a list near zero says nothing, and the sample comes out as a mix of all classes.
- **Every training step also applies weight decay**: each number is pulled a little toward zero. It is meant as a mild brake against over-fitting.
- **A client that has class c** sees it in some of its batches. Each time, training pushes list c to describe class c better, which in practice makes it longer. **A client that does not have class c** never sees it, so the only thing that ever touches list c on that client is the pull toward zero.
- **The optimiser turns the mild brake into a full step.** Adam divides each update by the typical size of that number's recent updates. A number that only ever receives the small, steady pull is divided by that same small size, so the pull becomes a step as large as real learning. The size of the decay no longer matters, only whether it is there.
- **Rare classes lose the tug-of-war.** A rare class appears in few batches even on the clients that have it (MNIST's rarest class has 24 samples in the whole federation). Its pushes are small and occasional; the pulls from clients without it come at full size on every step.

- **The server averages all clients' lists**, pulls outweigh pushes, and list c ends every round shorter than it began. After a few dozen rounds the rare lists are close to zero.
- **The exact condition (Theorem 1, Part 2.2):** list c shrinks unless the share of clients holding class c is above θ = loss ÷ (gain + loss). Because rare classes gain little, θ is 0.61–0.93: a rare class needs 61–93 % of clients to survive. On MNIST the rarest classes are held by 7 or 8 of 10 clients and are still erased.

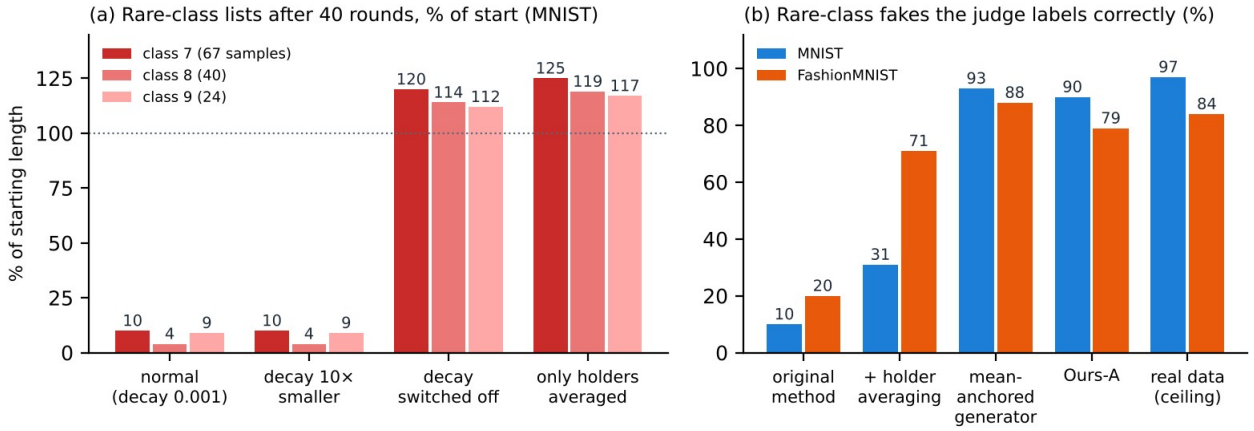


Figure 16. (a) One change at a time on the real generator. (b) Share of rare-class fakes that a judge trained only on real data labels correctly.

How we know. We trained the real generator four times, changing one thing each time (Figure 16a). Normal training: the three rarest lists end at 4–10 % of their starting length. Decay ten times smaller: *identical* curves. Decay switched off: they grow to 112–120 %. Only clients that hold the class averaged: they grow to 117–125 %. The consequence shows in the fakes (Figure 16b): a judge trained on real data recognises only 10 % (MNIST) and 20 % (FashionMNIST) of the original method's rare-class fakes, against 97 % and 84 % of real samples.

If the explanation were wrong. If rare classes simply had too little data, switching off the decay would not help, but it restores them fully. If the decay acted as a mild brake (as with plain gradient descent), ten times less decay would mean ten times slower erasing, but the curves are identical, as the optimiser explanation predicts. If the half-the-clients rule of version 1 were right, classes held by 7–9 of 10 clients would be safe, but they are erased.

The fix, and why it removes the cause.

- **Holder-weighted averaging** (first fix): when the server averages list c, only clients that have class c vote, weighted by how many samples they have. The clients that could only pull toward zero no longer count, so the loss term of Theorem 1 is gone and the list can only grow by what holders teach it. Result: rare lists 117–125 % of start; judge recognises 31 % / 71 % of rare fakes; +9.0 points (MNIST), +11.2 (FashionMNIST).
- **Mean-anchored generator** (final fix): removes class lists altogether. The class enters as its exact average feature, which each client sends once as a sum (secure aggregation: the server sees only the total). A sum cannot shrink. The generator's own weights are shared by all classes and learn only *how a sample differs from its class average*, which every client trains on every class it has. Result: the judge recognises 93 % / 88 % of rare fakes.
- **Why the anchor beats holder averaging:** holder averaging still has to *learn* each list from the few rare samples; the class average is exact from the first round.

How sure. The bound is proven (Appendix A). The cause is tested by changing one thing at a time on the real generator (1 seed; the bound itself is exact). The accuracy consequences hold over 3 seeds and four datasets.

7.2 The fakes are in the right place but too narrow and too clean

What goes wrong. After the mean anchor, fakes sit around the right class average, but they are bunched far too tightly: their spread is 0.47 (MNIST) and 0.45 (FashionMNIST) of the real spread. And their classes are separated much more cleanly than real classes: separability 0.78 / 1.35 against 0.30 / 0.60 for real data.

The cause, step by step.

- The generator (a variational autoencoder) learns by rebuilding each real feature from a compressed code. When it is unsure, the output with the lowest error is the *average* of the plausible answers, and averaging removes the sample-to-sample variety.
- Its random input is meant to put the variety back, but in practice it is under-used. The result is "textbook" samples: correct, typical, too similar to each other.
- **Why that hurts:** a model trained on textbook samples draws its class boundaries to fit tight, well-separated clusters. Real samples spread further and overlap more, so the ones near the edge of a class fall on the wrong side, like learning handwriting only from neat examples and then reading a doctor's note.

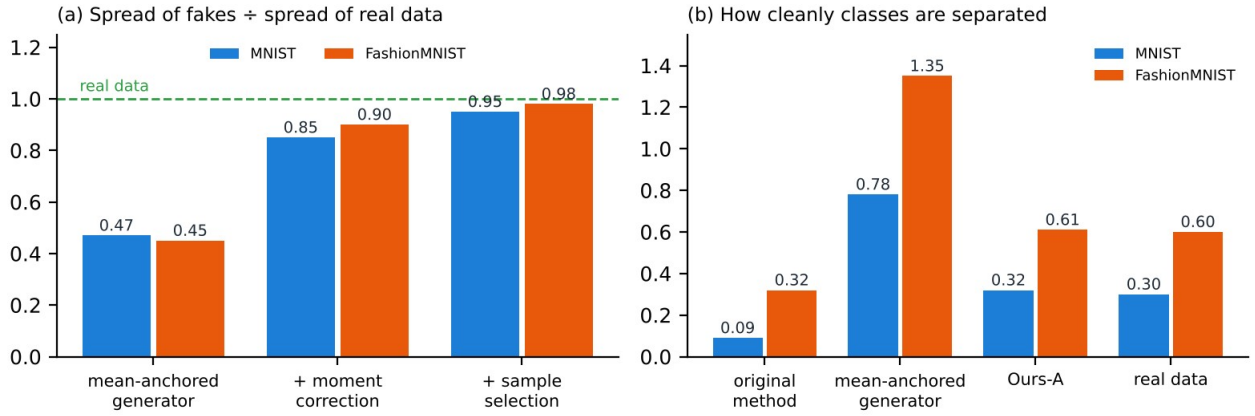


Figure 17. (a) Spread of fake features divided by the spread of real ones (1 = right). (b) How cleanly the fake classes are separated, against real data.

How we know. Spread and separability are measured directly against the real features (Figure 17). In the leave-one-out ablation, removing moment correction cuts the rare-class spread from 0.93 to 0.59 (MNIST) and costs 0.5–1.0 points, while the judge's recognition of the fakes does *not* fall (FashionMNIST 83.9 vs 79.2). So moment correction does not help by making fakes more recognisable; it helps by giving them the right variety.

If the explanation were wrong. If narrowness did not matter, correcting only the spread and centre would not change accuracy. It adds 1.6 points on both datasets, and the gain comes with the spread returning to that of real data, not with any change in recognisability.

The fix, and why it removes the cause. Moment correction: for each class, measure the fakes' average and spread (500 samples), then shift every fake so its average equals the exact real average and stretch it so its spread equals the exact real spread (both from the uploaded sums). Of all ways to reach the right average and spread, this one moves the samples the least (Proposition 3), so it keeps whatever shape the generator learned and corrects only the two numbers that were wrong. No training, two numbers per class.

Result. Spread 0.85 / 0.90; separability 0.32 / 0.61, matching real data (0.30 / 0.60); fake-to-real distance 0.20 → 0.089 (MNIST); +1.6 points on both datasets. **How sure:** measured on 3 seeds; the minimum-change property is proven.

7.3 Right centre and spread, but the wrong shape

What goes wrong. Two clouds of points can have the same centre and the same spread and still look different: one may have a sub-cluster the other lacks, or miss a style (for example sevens written with a cross-bar).

The cause. The centre and the spread are only two numbers per class; the shape needs many more.

The fix, and why it works. Sample selection: each client also sends, once, a sum of *fingerprints* of its features (6,144 random features that together describe the shape of a class). The server draws four times as many fakes as it needs and keeps the subset whose average fingerprint is closest to the real one: at each step it adds the candidate that best fills what is still missing. A known bound says a model's error on real data is at most its error on fakes plus a constant times the fake-to-real distance (Lemma 1); selection lowers exactly that distance.

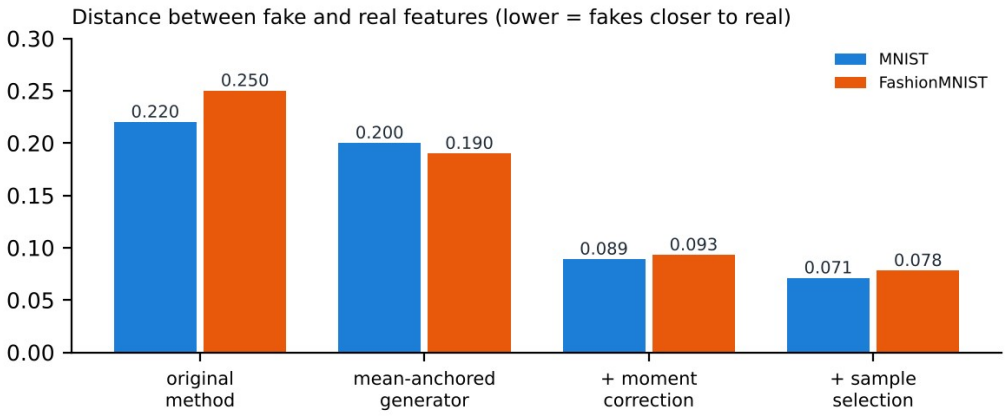


Figure 18. Distance between fake and real features after each step (lower is better).

How we know. The distance falls from 0.089 to 0.071 (MNIST) and 0.093 to 0.078 (FashionMNIST). Across all runs, lower distance goes with higher real accuracy (correlation −0.69 and −0.47). In the full method, selection helped in 9 of 9 paired tests (+0.7 to +0.8 points).

What we cannot yet explain. In the simpler test (train a model only on fakes, test on real data) selection adds nothing (−0.2 / −1.1). Its gain appears only together with the real-data epochs and consensus labels. It is also the smallest part in the ablation (−0.3 to −0.7 when removed). **How sure:** the effect is consistent but small, and its exact route is open.

7.4 Each model leans toward its client's common classes

What goes wrong. A client's model predicts the classes that are common *on that client* far too often, and the client's rare classes too rarely.

The cause, step by step.

- Every round ends with 5 epochs on the client's own real data. If 90 % of that data is one class, the cheapest way to lower the training error is to predict that class more often.
- The balanced fake epochs at the start of the round are partly undone by these 5 skewed epochs, every round.

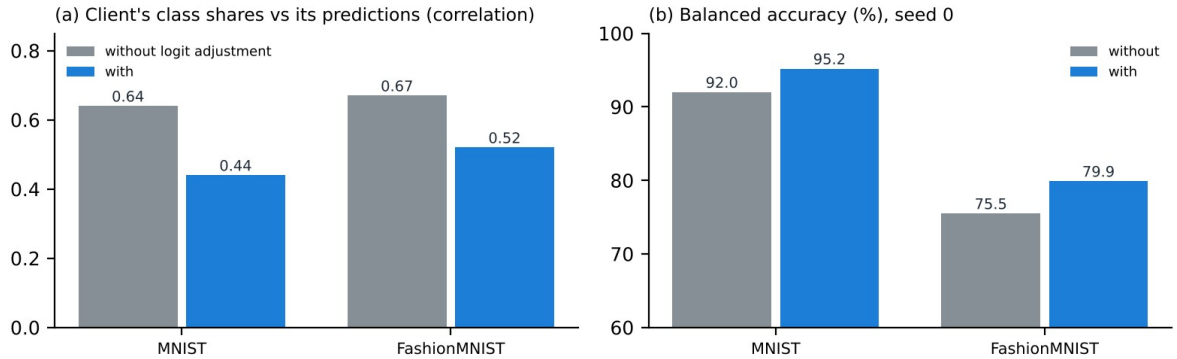


Figure 19. (a) Correlation between how often a client sees each class and how often its model predicts it (0 = no lean). (b) Balanced accuracy with and without logit adjustment.

How we know. The correlation between a client's class shares and its model's predictions is 0.64 (MNIST) and 0.67 (FashionMNIST): models predict what their client has.

The fix, and why it removes the cause. Logit adjustment: during the real-data epochs, add the logarithm of the class's share on that client to the model's score for that class before computing the loss. A common class already starts ahead in the score, so the model gains nothing by leaning toward it and stops learning the lean. At test time the head start is removed and the scores are balanced. The client's class shares never leave the client.

Result. Correlation 0.44 / 0.52; balanced accuracy 92.0 → 95.2 (MNIST) and 75.5 → 79.9 (FashionMNIST); the largest single part in the ablation (−3.0 / −6.2 points when removed). **Note:** correcting the lean after training (bias calibration) recovers most of this too: the job is "remove the client's skew", and either tool does it. **How sure:** measured on 3 seeds; an established technique (Menon et al., 2021).

7.5 Ambiguous fakes get a confident label

What goes wrong. Every fake comes with the label it was drawn for. Some fakes look like two classes (a shirt halfway between a T-shirt and a pullover). A hard "100 % T-shirt" label teaches the model to be confidently wrong near the boundary between the two.

The fix, and why it works. Consensus labels: the training target becomes half the drawn label plus half the average prediction of the 10 different model types on that same fake. Different architectures make different mistakes, so their average is a better estimate of what the sample really looks like. Clients send only sums of predictions on a shared pool of fakes.

How we know. Compared with a judge trained on real data, the mixed target is closer than the hard label in every class (total-variation distance 0.100 → 0.092 on MNIST, 0.212 → 0.202 on FashionMNIST). Removing consensus labels costs 1.4–1.6 points (FashionMNIST $p = 0.008$).

If the explanation were wrong. If agreement between models were good evidence of a correct label, trusting it more would help. A Bayesian version that trusted agreement more did *worse* (83.2 vs 84.2): models trained on the same generator agree with the generator, so their agreement is partly circular. Half-and-half worked better than full trust. **How sure:** measured on 3 seeds; the gain is medium-sized and consistent.

7.6 More training on fakes helps only once the fakes are good

What goes wrong. The original method trains 1 fake epoch per round. More fake training looks like an easy win, but with the original generator it adds nothing.

The cause. A model's error has two parts: noise from having little real data, which falls as you add fake data, and bias from fakes that are wrong, which grows the more you rely on them. The best share of fake training falls as the fakes get worse (Proposition 5). With bad fakes, extra fake epochs add bias as fast as they remove noise.

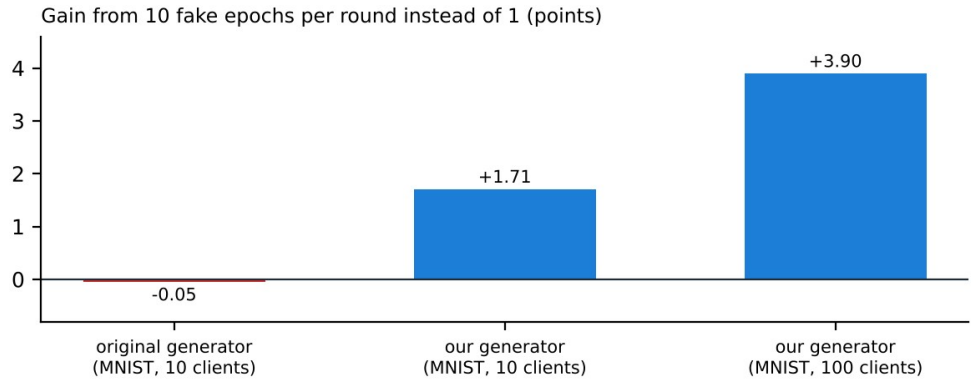


Figure 20. Gain from 10 fake epochs per round instead of 1.

How we know. Exactly as predicted: −0.05 points with the original generator, +1.7 with ours (MNIST long tail), +3.9 with 100 clients, where each client has less real data and the noise part is larger. **Limit:** on CIFAR-10 and SVHN, where fakes are still worse, 10 epochs give a higher peak but a lower final accuracy than 1. **How sure:** proven model, measured on 3 seeds.

7.7 Why the main method is weak on SVHN

What goes wrong. On SVHN, Ours-A with the mean anchor reaches only 54.4, below the simpler holder-weighted averaging plus logit adjustment (62.5).

The cause, step by step.

- The mean-anchored generator tells classes apart *only* through their class averages: two generated classes can differ at most as much as their averages do.
- SVHN digits come with varying colours, backgrounds and neighbouring digits. After the frozen feature extractor, the average feature of a "3" is almost the same as that of an "8".
- So the generator receives nearly the same instruction for every class, and its fakes cannot be told apart by class.

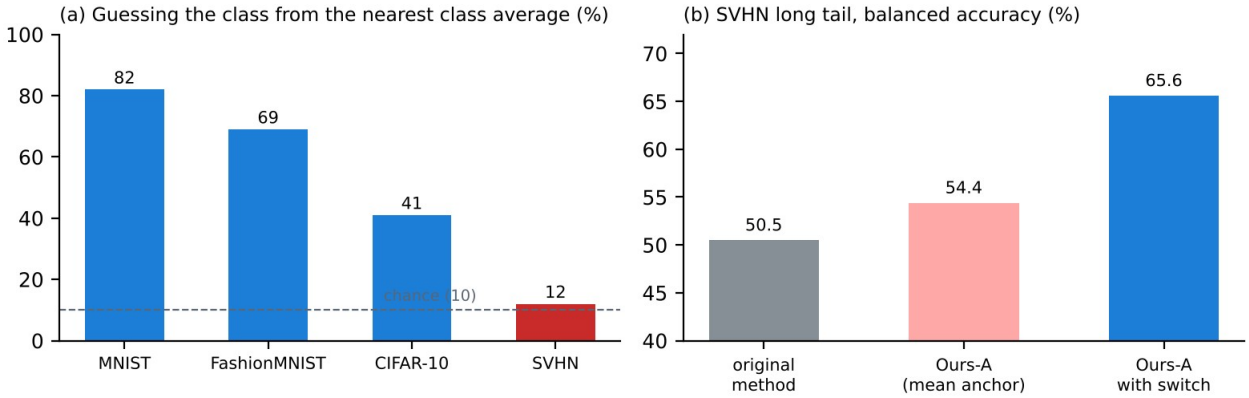


Figure 21. (a) Guessing each real sample's class from the nearest class average: high on MNIST, near chance on SVHN. (b) SVHN long tail: the automatic switch fixes it.

How we know. Guessing a real sample's class from the nearest class average works 82 % of the time on MNIST, 69 % on FashionMNIST, 41 % on CIFAR-10, and only 12 % on SVHN (chance is 10 %). The prediction is that the anchor fails exactly where this is near chance, and it fails only on SVHN.

The fix. The server computes how well the class averages separate the classes, from sums it already has. Below a threshold (separability 0.05) it keeps the original class lists, protected by holder-weighted averaging, and still applies moment correction and sample selection. Result: 65.6 under the long tail (+15.1 over the original method) and 77.07 in the balanced setting (above the paper's 76.26). The other three datasets are above the threshold and unaffected. **How sure:** measured on 3 seeds; the threshold was chosen from these four datasets and needs a fifth dataset to be confirmed.

7.8 What still fails: the last layer

What goes wrong. Even with Ours-A, each model's last layer loses some accuracy compared with the best last layer for the same model (Part 6): 0.1 points on MNIST, 1.9 on FashionMNIST, 6.2 on CIFAR-10. It does not shrink with more clients.

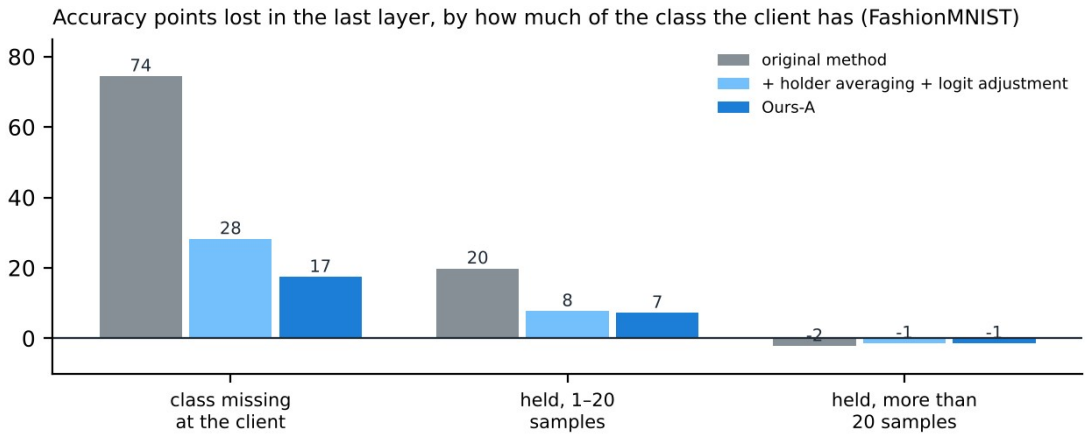


Figure 22. Accuracy lost in the last layer, split by how much of the class the model's own client has (FashionMNIST, 1 seed, shortened schedule).

Where it sits (measured). Almost entirely on classes the model's own client does **not have**: there Ours-A loses 17 points (the original method 74). Classes the client has with 1–20 samples lose 7; classes it has with more than 20 samples lose nothing.

The cause, as far as we know.

- For a class its client lacks, a model's last layer learns that class **only from fakes**. For classes the client has, it also learns from real samples in the last 5 epochs of every round.
- So the weights for missing classes end up the **wrong size** relative to the others, and those classes are predicted too rarely. Rescaling all class weights to the same size recovers 16–34 % of the loss: this explains about a quarter.
- **The rest is not explained yet.** Our first explanation, that the fake class averages are slightly off inside each model, was tested and was wrong: it did not predict which classes lose the most (correlation 0.10–0.33, against 0.33–0.90 for the simple "how much of the class came from fakes").

What fixes it anyway. The exact readout (Part 6.1) sets the last layer from exact sums of real features and never sees a fake. It lands within 0.8 points of the best possible last layer. **How sure:** the location of the error and the fix are measured (1 seed, shortened schedule; consistent with the 3-seed numbers of Part 6); the cause is only a quarter explained.

7.9 Why the dropped ideas failed

Idea	Why it failed
Server-side ensemble distillation	It teaches each model on fakes. It cannot pass on anything the fakes do not contain, so it is limited by the same fake-to-real gap (equation 3).
Re-training the last layer on balanced fakes	Same limit, and the fakes are worse than real data: weaker than logit adjustment (62.0 vs 65.2). The exact readout works because it uses real sums instead of fakes.
Gaussian generator from exact statistics	A model trained on its samples becomes a straight-line classifier. Straight lines work on MNIST but reach only 21 % on SVHN features.
Bayesian consensus labels	Trusted model agreement more; agreement is partly circular (7.5).
Filling each real batch's missing classes with fakes	Needs good rare-class fakes, which did not exist before the generator was fixed.
Feature mixing (MixUp and our variant)	Lowered accuracy (59.98 vs 60.93). Mixing features after a frozen extractor cannot add the image-level variety that the paper's image mixing adds while training the whole model.
Lazy decay (decay only the rows of classes in the batch)	Removes the same pull toward zero as holder-weighted averaging, so on top of it adds nothing (78.6 vs 79.1).

Limitations

- SVHN: class means carry no class information, so Ours-A is below GeFL-F in the balanced setting (74.4 vs 75.8).
- Balanced data: gains are +1 to +2 points; the method is built for imbalance.
- The frozen feature extractor limits every method (CIFAR-10 oracle ≈ 51 under the long tail).
- On CIFAR-10 and SVHN, 10 fake epochs give a higher peak but a lower final accuracy than 1.
- Against a known last-layer recalibration of GeFL-F, the margin is large only on MNIST (+4.1); on FashionMNIST it is +0.5 (Part 6).
- Per-component significance needs 2 more seeds (K14); the collapse test T01 has 1 seed (the bound itself is exact).
- Two audit points to state: averaging BatchNorm running variances drops the between-client part of the variance; best_mean_acc selects the best round on the test set.

8. Status and next steps

Item	Why	Status
Automatic switch to class rows when class means are weak	fix SVHN	**done**: SVHN 54.4 → 65.6 (long tail), 77.07 (balanced)
Leave-one-out ablation (K09)	prove each part is needed	**done**: 30/30 negative; per-part significance needs K14
FashionMNIST at 50 / 100 clients	complete the grid	**done**: 79.5 / 79.0
Collapse theorem rewritten and tested (T01)	the version-1 rule was wrong	**done** (1 seed)
Last-layer analysis and exact readout (T03)	where the remaining error is	**done** (1 seed, short schedule); mechanism open
K14: ablation seeds 3–4 and covariance correction (Kaggle, ≈ 4 GPU hours)	per-part significance; covariance correction stability	notebooks ready, not run
K15: existing collapse fixes (AdamW, no decay on rows, lazy decay) vs holder-weighted averaging at 10/50/100 clients (≈ 7 GPU hours)	the first question a reviewer will ask	notebooks ready, not run
CIFAR-10 / SVHN long tail with exact readout logged (≈ 6 GPU hours)	GeFL-F + recalibration baseline on the hard datasets	proposed; needs approval (re-runs existing settings)
Covariance correction covariance correction	exact covariance where data suffice	FashionMNIST: 81.57 (long tail), 85.75 (balanced); MNIST unstable on one seed
Paper's other federated learning baselines under the long tail	complete comparisons	optional

Appendix A. Proofs

Theorem 1. Let r_k be client k's row after local training and $r^+ = (1/K)\,\Sigma_k\, r_k$. Define $G_k = \|r_k\| - \|r\|$ for holders and $D_k = \|r\| - \|r_k\|$ for non-holders. By the triangle inequality $\|r^+\| \le (1/K)\,\Sigma_k\,\|r_k\| = \|r\| + p\,\bar{G} - (1-p)\,\bar{D}$, with equality if and only if all r_k point the same way. The right side is below $\|r\|$ exactly when $p < \bar{D} / (\bar{G} + \bar{D})$. ∎

Lemma 2. A non-holder's gradient on r_c is exactly $\lambda\, r_c$. Adam's step is $\eta \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$; multiplying every gradient by λ multiplies $\hat{m}$ and $\sqrt{\hat{v}}$ by λ, so the step is unchanged as long as $\lambda|r| \gg \varepsilon$. Hence $\bar{D}$ does not depend on $\lambda > 0$, and $\bar{D} = 0$ at $\lambda = 0$. Under stochastic gradient descent the same step would be $\eta\,\lambda\,\|r\|$. ∎

Proposition 3. With $Y = T(X)$, $E\|Y - X\|^2 = V_c + \tilde{V}_c + \|m_c - \tilde{m}_c\|^2 - 2E\langle Y - m_c, X - \tilde{m}_c\rangle$. By Cauchy–Schwarz the last term is at most $\sqrt{(V_c\,\tilde{V}_c)}$, with equality if and only if $Y - m_c = \sqrt{(V_c/\tilde{V}_c)}(X - \tilde{m}_c)$. ∎

Proposition 5. With $w = \lambda n_s/(n_r + \lambda n_s)$, the error becomes $\sigma^2(1-w)/n_r + w^2 b^2$, a convex parabola minimised at $w^* = \sigma^2/(2n_r b^2)$. The optimum is clipped to the feasible range, so the optimal synthetic share is non-increasing in b and reaches its maximum as $b \to 0$. ∎

Lemma 1 is the standard kernel bound: $|E_G\,\ell - E_P\,\ell| = |\langle \ell, \mu_G - \mu_P\rangle| \le \|\ell\| \cdot$ fake-to-real distance(G, P).

Appendix B. Hyperparameters and experiment inventory

Dataset	Fraction / train size	Batch	T_FE / T_KA / T_TN	Latent l	Feature h
MNIST	0.1 / 6,000	64	20 / 100 / 50	16	3×16×16
FashionMNIST	0.1 / 6,000	64	20 / 100 / 50	16	3×16×16
SVHN	0.1 / 7,326	64	50 / 100 / 100	16	3×16×16
CIFAR-10	0.5 / 25,000	128	50 / 200 / 100	50	10×16×16

Ours-A settings: 6,144 random Fourier features; herding pool 4n; consensus weight $\beta = 0.5$; logit adjustment temperature $\tau = 1$; 10 synthetic epochs, $T_r = 5$.

Experiment series	Question answered
F01, F10, E20 (local)	Main long-tail comparison and the first anchored generator
E16, E21, E22 (local)	Synthetic budget (Proposition 5); 100 clients
E24, E25 (local)	Kernel herding, server-trained generator (kernel matching), the anchored stack
E26, E27, E29, E31, E32 (local)	Formal differential privacy; balanced at 50/100 clients; hybrid
K01–K04 (Kaggle)	CIFAR-10 and SVHN baselines; regimes, privacy, ablations; client scaling; diffusion generator
K08, K10, K11, K13 (Kaggle)	Ours-A on CIFAR-10 and SVHN; SVHN switch; hybrid; our mixing variant vs MixUp
K09 (Kaggle)	Leave-one-out ablation of Ours-A
E32 (local)	Covariance correction, covariance version of the moment fix
T01, T03 (local CPU, pre-registered)	Collapse theorem on the real generator; last-layer gap and exact readout
K14, K15 (Kaggle, ready, not run)	Ablation seeds 3–4; existing collapse fixes vs holder-weighted averaging

All code, logs and per-seed results are in the project repository (run ledger: experiments/LEDGER.md), and every number in this report can be regenerated from the logged runs.